Résumé

Ce projet de fin d'études a été réalisé au sein de Horizon Education dans le cadre du développement d'une solution éducative destinée aux jeunes enfants. L'objectif principal est de concevoir et de développer un jeu éducatif permettant d'initier les enfants aux concepts fondamentaux de l'Intelligence Artificielle à travers une série de mini-jeux interactifs et progressifs.

Le jeu est intégré à la plateforme éducative Ferid Around the World et couvre les principales étapes du développement d'un système d'Intelligence Artificielle, notamment la collecte des données, le nettoyage des données, la reconnaissance d'images, la détection de visages, le débogage des systèmes d'IA, la planification de trajectoires et la prise de décision. Chaque activité a été conçue selon une approche ludique afin de rendre ces notions accessibles aux jeunes apprenants.

Le projet a été développé avec le moteur de jeu Unity en utilisant le langage C#, tout en suivant la méthodologie Agile Scrum afin d'assurer une gestion itérative du développement. En complément des mini-jeux éducatifs, une fonctionnalité basée sur l'Intelligence Artificielle a été intégrée grâce à un modèle de reconnaissance vocale développé avec PyTorch et déployé dans Unity via Sentis, permettant au joueur de contrôler un robot à l'aide de commandes vocales en anglais.

Les résultats obtenus démontrent la faisabilité de l'intégration de concepts d'Intelligence Artificielle dans un environnement éducatif interactif adapté aux enfants. Cette application constitue une base évolutive pouvant être enrichie par de nouveaux contenus pédagogiques ainsi que par l'ajout du support multilingue et l'amélioration des performances du modèle de reconnaissance vocale.

Mots-clés : Intelligence Artificielle, Jeu éducatif, Unity, C#, Scrum, Apprentissage par le jeu, Reconnaissance vocale, Réseaux de neurones convolutifs, PyTorch.

Abstract

This Final-Year Project was carried out at Horizon Education with the objective of designing and developing an educational game that introduces young children to the fundamental concepts of Artificial Intelligence through interactive and engaging gameplay.

The developed game is integrated into the Ferid Around the World educational platform and is organized as a sequence of educational mini-games representing the main stages of the Artificial Intelligence development process, including data gathering, data cleaning, image recognition, face detection, AI system debugging, robot path planning, and decision making. Each activity was designed using age-appropriate interactions, visual feedback, and progressive learning mechanics to simplify complex AI concepts.

The application was developed using the Unity game engine and the C# programming language while following the Agile Scrum methodology to ensure iterative development and continuous integration. In addition to the educational gameplay, the project incorporates an Artificial Intelligence speech recognition module based on a Convolutional Neural Network (CNN) developed with PyTorch and deployed in Unity using Sentis. This module enables learners to control a virtual robot through English voice commands, providing practical exposure to a real AI application.

The obtained results demonstrate the effectiveness of combining educational games with Artificial Intelligence technologies to create an engaging learning environment for children. The developed solution provides a scalable foundation that can be extended through multilingual speech recognition, improved model performance, and additional educational activities covering more advanced AI concepts.

Keywords: Artificial Intelligence, Educational Game, Unity, C#, Scrum, Gamification, Speech Recognition, Convolutional Neural Network, PyTorch.

TABLE OF CONTENTS

Abstract i
Acknowledgments ii
Table of Contents iii
List of Figures v
List of Tables vii

General Introduction 1
Chapter 1 General Project Context 2
Introduction 3
1.1 Presentation of the Host Organization 3
1.1.1 Horizon Education 3
1.1.2 Ferid Around the World 4
1.2 Project Presentation 5
1.2.1 Existing Solutions 6
1.2.2 Problem Statement 6
1.2.3 Proposed Solution 7
1.3 Requirements Analysis 7
1.3.1 Functional Requirements 8
1.3.2 Non-Functional Requirements 9
1.4 General Use Case Diagram 10
Conclusion 11

Chapter 2 System Architecture, Development Technologies and Project Management Methodology 12
Introduction 13
2.1 Game Architecture 13
2.1.1 Overall System Architecture 13
2.1.2 AI Learning Architecture 14
2.2 Technologies Used 15

2.2.1 Unity 15
2.2.2 C# 16
2.2.3 Visual Studio 16
2.2.4 Krita 16
2.2.5 Git 17
2.2.6 GitHub 17
2.2.7 ClickUp 17
2.2.8 Slack 18
2.3 Project Management Methodology 18
2.3.1 Scrum Methodology 18
2.3.2 Scrum Roles 19
2.3.3 Scrum Artifacts 20
2.3.4 Scrum Events 21
2.3.5 Application of Scrum to the Project 22
2.4 Product Backlog 22
2.5 Sprint Planning 23
2.6 Artificial Intelligence Module 24
2.6.1 Introduction 24
2.6.2 Artificial Intelligence Domain 24
2.6.3 Deep Learning Model 25
2.6.4 Model Selection 26
2.6.5 Dataset Preparation 27
2.6.6 Audio Preprocessing 27
2.6.7 Model Training 28
2.6.8 Integration with the Educational Application 29
2.6.9 Current Status and Future Enhancements 30
Conclusion 30

Chapter 3 Sprint 1 – Sprint 3 31
Introduction 32
3.1 Sprint 1: Data Gathering 32
3.2 Sprint 2: Data Cleaning 36
3.3 Sprint 3: Image Recognition Evaluation 40

3.4 UML Modeling 44
3.4.1 Use Case Diagram 44
3.4.2 Sequence Diagram 45
3.4.3 Class Diagram 46
Conclusion 46

Chapter 4 Sprint 4 – Sprint 6 47
Introduction 48
4.1 Sprint 4: Face Detection 48
4.2 Sprint 5: AI Agent Debugging 52
4.3 Sprint 6: Robot Path Planning 56
4.4 UML Modeling 60
4.4.1 Use Case Diagram 60
4.4.2 Sequence Diagram 61
4.4.3 Class Diagram 62
Conclusion 62

Chapter 5 Sprint 7 – Sprint 8 63
Introduction 64
5.1 Sprint 7: AI Decision Making 64
5.2 Sprint 8: Game Map 68
5.3 UML Modeling 72
5.3.1 Use Case Diagram 72
5.3.2 Sequence Diagram 73
5.3.3 Class Diagram 74
Conclusion 75

General Conclusion 76

Webography.....77

LIST OF FIGURES

Figure 1.1 Horizon Education logo 4
Figure 1.2 Ferid Around the World Application 5
Figure 1.3 General Use Case Diagram 10

Figure 2.1 Overall System Architecture 13
Figure 2.2 AI Learning Architecture 14
Figure 2.3 Unity Logo 15
Figure 2.4 C# Logo 16
Figure 2.5 Visual Studio Logo 16
Figure 2.6 Krita Logo 16
Figure 2.7 Git Logo 17
Figure 2.8 GitHub Logo 17
Figure 2.9 ClickUp Logo 17
Figure 2.10 Slack Logo 18
Figure 2.11 Artificial Intelligence hierarchy 25
Figure 2.12 VoiceCNN Processing Architecture 26
Figure 2.13 Audio Preprocessing Pipeline 28
Figure 2.14 Model Training Workflow 29
Figure 2.15 Integration with Unity 30

Figure 3.1 Activity Workflow Data Gathering 33
Figure 3.2 Data Gathering Tutorial 34
Figure 3.3 Data Gathering Gameplay 34
Figure 3.4 Correct Object Classification 35
Figure 3.5 Success Panel 35
Figure 3.6 Activity Workflow Data Cleaning 37
Figure 3.7 Data Cleaning Tutorial 38
Figure 3.8 Data Cleaning Gameplay 38
Figure 3.9 Wrong Classification 39
Figure 3.10 Activity Workflow Image Recognition 41

Figure 3.11 Image Recognition Tutorial 42
Figure 3.12 Image Recognition Gameplay 42
Figure 3.13 Evaluation Screen 43
Figure 3.14 Use Case Diagram 44
Figure 3.15 Sequence Diagram 45
Figure 3.16 Class Diagram 46

Figure 4.1 Activity Workflow Face Detection 49
Figure 4.2 Face Detection Gameplay 50
Figure 4.3 Evaluation Example 50
Figure 4.4 Correcting Robot's Answer 51
Figure 4.5 Activity Workflow AI Agent Debugging 53
Figure 4.6 AI Agent Debugging Gameplay 54
Figure 4.7 Indicate Missed Agents 54
Figure 4.8 Agent Debugging Progress 55
Figure 4.9 Activity Workflow Robot Path Planning 57
Figure 4.10 Robot Path Planning Gameplay 58
Figure 4.11 Placing Action Cards 58
Figure 4.12 Submit Selection 59
Figure 4.13 Use Case Diagram 60
Figure 4.14 Sequence Diagram 61
Figure 4.15 Class Diagram 62

Figure 5.1 Activity Workflow AI Decision Making 65
Figure 5.2 AI Decision Making Gameplay 67
Figure 5.3 Action Cards Arrangement 67
Figure 5.4 Activity Workflow Game Map 69
Figure 5.5 Locked and Unlocked Activities 70
Figure 5.6 Locked and Unlocked Levels 70
Figure 5.7 Game Map Interface 71
Figure 5.8 Use Case Diagram 72
Figure 5.9 Sequence Diagram 73
Figure 5.10 Class Diagram 74

LIST OF TABLES

Table 1.1 AI Learning Journey through Mini-Games 5
Table 1.2 Functional Requirements 8
Table 1.3 Non-Functional Requirements 9

Table 2.1 AI Learning Mapping Across Domains 14
Table 2.2 Scrum Roles and Responsibilities 20
Table 2.3 Product Backlog 22
Table 2.4 Sprint Planning 23
Table 2.5 Justification for Selecting CNN 27
Table 2.6 Voice Command Categories 27

Table 3.1 Sprint 1 Backlog 32
Table 3.2 Sprint 2 Backlog 36
Table 3.3 Sprint 3 Backlog 40

Table 4.1 Sprint 4 Backlog 48
Table 4.2 Sprint 5 Backlog 52
Table 4.3 Sprint 6 Backlog 56

Table 5.1 Sprint 7 Backlog 64
Table 5.2 Sprint 8 Backlog 68

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ASR	Automatic Speech Recognition
CNN	Convolutional Neural Network
CPU	Central Processing Unit
FFT	Fast Fourier Transform
GUI	Graphical User Interface
IDE	Integrated Development Environment
LSTM	Long Short-Term Memory
ONNX	Open Neural Network Exchange
RNN	Recurrent Neural Network
UI	User Interface
UML	Unified Modeling Language

General Introduction

Artificial Intelligence (AI) has become one of the most influential technologies of the modern era, transforming numerous aspects of everyday life, including healthcare, transportation, communication, and education. As intelligent systems become increasingly integrated into society, understanding the fundamental principles of Artificial Intelligence is becoming an essential skill for future generations. However, most educational resources dedicated to AI are designed for older learners and often require programming knowledge or mathematical concepts that are difficult for young children to understand.

Educational games have emerged as an effective solution for simplifying complex concepts by combining learning with interactive and engaging activities. Through gamification, children can acquire knowledge naturally by experimenting, solving problems, and receiving immediate feedback. This pedagogical approach increases motivation while improving understanding and knowledge retention.

This Final-Year Project was carried out at Horizon Education, an educational technology company specializing in the development of digital learning solutions for children. The objective of the project is to design and develop an educational game that introduces children to the fundamental concepts of Artificial Intelligence through a series of interactive mini-games. The developed game is integrated into the Ferid Around the World educational platform and guides learners through the main stages involved in building an intelligent system, including data gathering, data cleaning, image recognition, face detection, AI system debugging, robot path planning, and decision making.

Beyond the educational gameplay, the project also integrates a real Artificial Intelligence component based on speech recognition. A Convolutional Neural Network (CNN) developed using the PyTorch framework enables learners to interact with the application through English voice commands, providing practical exposure to an authentic AI technology while enriching the learning experience.

From a software engineering perspective, the project follows the Agile Scrum methodology, enabling iterative development, continuous testing, and progressive integration of new functionalities. The application was developed using Unity and C#, while additional technologies such as GitHub, ClickUp, Slack, and Visual Studio supported version control, project management, collaboration, and software development throughout the internship.

This report is organized into five chapters. The first chapter presents the general context of the project, the host organization, the problem statement, the proposed solution, and the requirements analysis. The second chapter describes the system architecture, the development technologies, the project management methodology,

and the Artificial Intelligence module integrated into the application. The third, fourth, and fifth chapters present the analysis, design, implementation, and UML modeling of the different educational mini-games developed during the successive Scrum sprints. Finally, the report concludes with an overall assessment of the project, the achieved results, and perspectives for future improvements.

Chapter 1

General Project Context

Introduction

Artificial Intelligence has become an essential technology in many aspects of everyday life, influencing how people communicate, learn, and solve problems. As AI continues to shape modern society, introducing its fundamental concepts at an early age has become increasingly important. However, most educational resources explain AI using technical terminology or programming concepts that are difficult for young children to understand.

Educational games provide an effective alternative by transforming abstract concepts into interactive experiences that promote exploration and active learning. Within this context, Horizon Education aims to enrich its educational platform by integrating innovative learning modules adapted to children's cognitive abilities.

This chapter presents the general context of the project. It begins with an overview of the host organization and its educational platform, followed by the presentation of the project, an analysis of existing educational solutions, the identified problem, and the proposed solution.

1.1 Presentation of the Host Organization

1.1.1 Horizon Education

The internship was conducted at Horizon Education, an educational technology company specialized in the development of interactive digital learning solutions for children. The company designs educational applications that combine modern technologies with pedagogical approaches to create engaging learning experiences.

Its activities focus on the development of serious games, educational mobile applications, interactive multimedia content, and digital learning platforms. By combining software engineering, game development, and educational design, Horizon Education seeks to make learning more engaging while encouraging curiosity, creativity, and critical thinking.

The company follows a learner-centered approach in which educational objectives are integrated into enjoyable and interactive experiences. This approach allows children to acquire new knowledge through exploration, experimentation, and problem-solving instead of relying solely on traditional teaching methods.



Figure 1.1. Horizon Education logo.

The company emblem used by Horizon Education across its products and communications.

1.1.2 Ferid Around the World

One of Horizon Education's flagship products is Ferid Around the World, an educational mobile application designed specifically for young children. The application serves as a digital learning platform that offers a collection of educational games and interactive activities covering multiple learning domains.

Rather than focusing on a single subject, the application introduces children to diverse topics such as mathematics, languages, science, geography, culture, environmental awareness, and life skills. Each activity is presented through colorful graphics, animations, voice guidance, and interactive gameplay adapted to children's age and learning pace.

The modular architecture of the application allows new educational content to be integrated continuously. This flexibility enables Horizon Education to expand the platform with additional learning experiences while maintaining a consistent user interface and educational methodology.

The project developed during this internship represents one of these educational modules. It extends the Ferid Around the World application by introducing a new learning experience dedicated to Artificial Intelligence, allowing children to discover AI concepts through interactive gameplay.



Figure 1.2. *Ferid Around the World Application*

The Ferid Around the World app icon, showing the robot mascot Ferid who guides learners through the platform.

1.2. Project Presentation

Artificial Intelligence is becoming an increasingly important part of everyday life, yet children rarely have access to educational resources that explain how intelligent systems work. Existing learning materials are generally designed for older students and often require programming knowledge or mathematical background.

The objective of this project is to develop a serious educational game that introduces children to the fundamental concepts of Artificial Intelligence through interactive mini-games. Instead of presenting theoretical explanations, the application enables children to learn by performing simplified AI-related tasks.

The game is integrated into the Ferid Around the World platform as a new educational module. It consists of a world map that guides children through seven sequential mini-games, each representing a fundamental stage of the Artificial Intelligence development process.

The educational journey includes the following concepts:

Table 1.1: AI Learning Journey through Mini-Games

Mini-game	AI Concept
Game 0	Data Gathering
Game 1	Data Cleaning
Game 2	Image Recognition
Game 3	Face Detection
Game 4	System Debugging
Game 5	Path Planning
Game 6	Decision Making

Maps each of the seven mini-games to the specific AI concept it teaches, from data gathering through to decision making.

Each mini-game introduces a specific AI concept through age-appropriate interactions, visual feedback, tutorials, and progressive difficulty, allowing children to understand the overall AI workflow while remaining engaged throughout the learning experience.

1.2.1 Existing Solutions

Several educational platforms have introduced children to Artificial Intelligence and computational thinking. Among the most popular is Machine Learning for Kids, which allows learners to build simple machine learning models using visual programming. While effective for introducing AI concepts, it primarily targets older children and requires basic programming knowledge.

Another widely used platform is Code.org AI for Oceans, which teaches AI classification through interactive activities focused on environmental protection. Although engaging, it covers only a limited aspect of Artificial Intelligence.

Google's Quick, Draw! demonstrates image recognition by allowing users to draw objects that an AI model attempts to identify. The application effectively illustrates computer vision but does not explain the broader AI development process.

Although these platforms provide valuable educational experiences, they generally focus on individual AI concepts rather than presenting the complete Artificial Intelligence pipeline in a structured and child-friendly manner.

1.2.2 Problem Statement

Despite the growing importance of Artificial Intelligence, few educational applications introduce AI concepts to young children in a comprehensive and accessible way. Most existing solutions either focus on programming, explain isolated AI techniques, or target older learners.

Consequently, children often lack an intuitive understanding of how Artificial Intelligence systems are developed, from collecting data to making autonomous decisions. This creates a need for an educational solution capable of simplifying the entire AI workflow through interactive activities adapted to children's cognitive abilities.

1.2.3 Proposed Solution

To address these limitations, this project proposes the development of an educational game integrated into the Ferid Around the World application.

The game introduces the complete Artificial Intelligence workflow through seven interconnected mini-games, each illustrating a fundamental AI stage. A world map organizes the learning progression, enabling children to navigate through the different concepts in a structured manner.

Rather than teaching programming or mathematical algorithms, the application emphasizes conceptual understanding through gameplay. Interactive tutorials, animations, visual feedback, and age-appropriate challenges encourage active participation while making complex AI concepts understandable and enjoyable for young learners.

1.3. Requirements Analysis

Requirements analysis is a fundamental phase of software development that identifies the expected functionalities and quality attributes of the proposed system. It establishes a clear understanding of the users' needs and serves as the foundation for the design and implementation phases.

The educational game developed during this internship aims to provide children with an engaging environment for learning Artificial Intelligence concepts through interactive gameplay. The system requirements are divided into functional requirements, which describe the services provided by the application, and non-functional requirements, which define the quality constraints that the system must satisfy.

1.3.1 Functional Requirements

The functional requirements describe the operations that the application must perform to achieve its educational objectives.

Table 1.2: Functional Requirements

ID	Functional Requirement
1	The system shall allow the player to start a new learning session.
2	The system shall display a world map that provides access to the available AI mini-games.
3	The system shall progressively unlock mini-games according to the player's progress.
4	The system shall provide interactive tutorials before introducing new gameplay

ID	Functional Requirement
	mechanics.
5	The system shall allow the player to complete educational activities related to Artificial Intelligence concepts.
6	The system shall evaluate the player's actions and provide immediate visual and audio feedback.
7	The system shall validate the successful completion of each level.
8	The system shall navigate the player back to the world map after completing a game or level.
9	The system shall manage level progression throughout the learning experience.
10	The system shall display animations, sound effects, and educational instructions during gameplay.

Lists the ten functional requirements the application must satisfy, covering session management, the world map, tutorials, gameplay evaluation, and progression.

1.3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes expected from the application.

Table 1.3: Non-Functional Requirements

Category	Requirement
Usability	The game shall provide an intuitive interface suitable for children between 5 and 10 years old.
Performance	Scene transitions and gameplay interactions shall be smooth and responsive.
Reliability	The application shall ensure stable execution without interrupting the learning experience.
Maintainability	The project shall adopt a modular architecture to facilitate future extensions and maintenance.
Scalability	New educational mini-games can be integrated into the platform without major architectural modifications.
Compatibility	The application shall be compatible with Android devices supported by Unity.
Accessibility	The interface shall use simple interactions, visual guidance, and audio instructions appropriate for young learners.

Lists the seven non-functional requirements covering usability, performance, reliability, maintainability, scalability, compatibility, and accessibility.

1.4. General Use Case Diagram

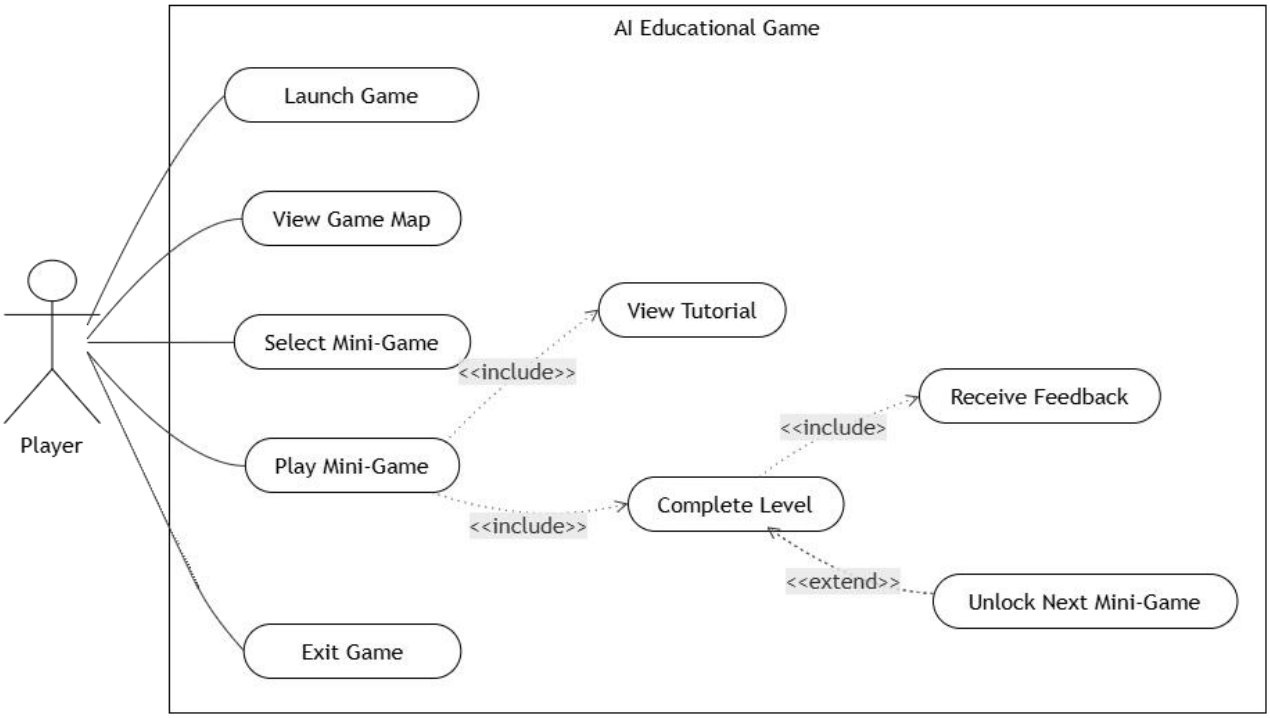


Figure 1.3: General Use Case Diagram

A use case diagram showing the player as the sole actor, able to launch the game, view the map, select and play a mini-game, view its tutorial, and receive feedback before unlocking the next activity.

The educational application involves a single primary actor: the Player, representing the child using the application. Through the world map, the player can access the available educational mini-games, complete learning activities, receive feedback, and progress through the different stages of the Artificial Intelligence learning journey.

The general use case diagram shown in Figure 1.3 illustrates the interactions between the player and the main functionalities offered by the system.

Conclusion

This chapter introduced the general context of the project by presenting the host company, the problem addressed, the proposed solution, and the project objectives. It also defined the functional and non-functional requirements that guided the development of the educational application. These elements establish the foundation for the design and implementation choices presented in the following chapters.

Chapter 2

System Architecture

Development Technologies

Project Management Methodology

Introduction

This chapter presents the technical organization adopted during the development of the educational game. It first describes the overall system architecture, followed by the development technologies and software tools used throughout the project. The chapter also introduces the Agile Scrum methodology adopted to manage the development process, the product backlog that defines the project requirements, the sprint planning established during the internship, and the research work conducted on integrating an Artificial Intelligence voice recognition model into the application.

2.1 Game Architecture

The educational application was designed using a modular architecture in which each mini-game represents a specific stage of the Artificial Intelligence learning process. The application is organized around a central Game Map that manages navigation and player progression while providing access to the different educational modules. This architecture promotes modularity, scalability, and maintainability, allowing new educational activities to be integrated with minimal modifications to the existing system.

2.1.1 Overall System Architecture

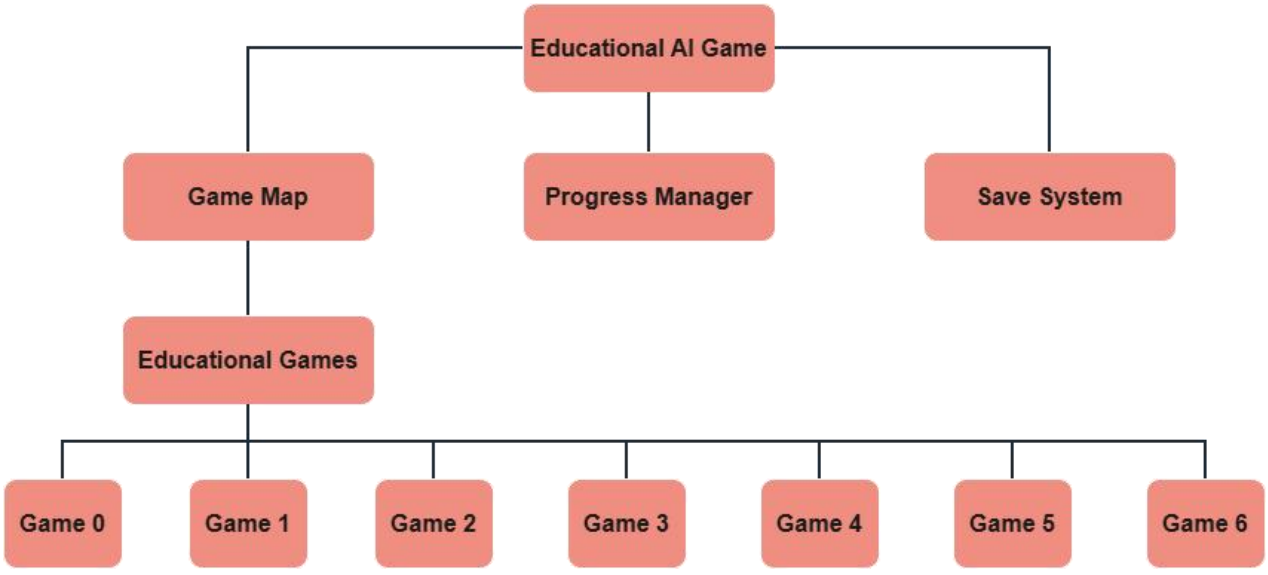


Figure 2.1: Overall System Architecture

A component diagram showing the application split into a Game Map, a Progress Manager, and a Save System, with the Game Map giving access to the seven educational mini-games.

Figure 2.1 illustrates the overall architecture of the educational application. The Game Map acts as the central navigation module, allowing players to access the different mini-games while managing progression and unlocked content. Each mini-game operates independently and focuses on a specific Artificial Intelligence concept, making the application modular and easy to extend.

2.1.2 AI Learning Architecture

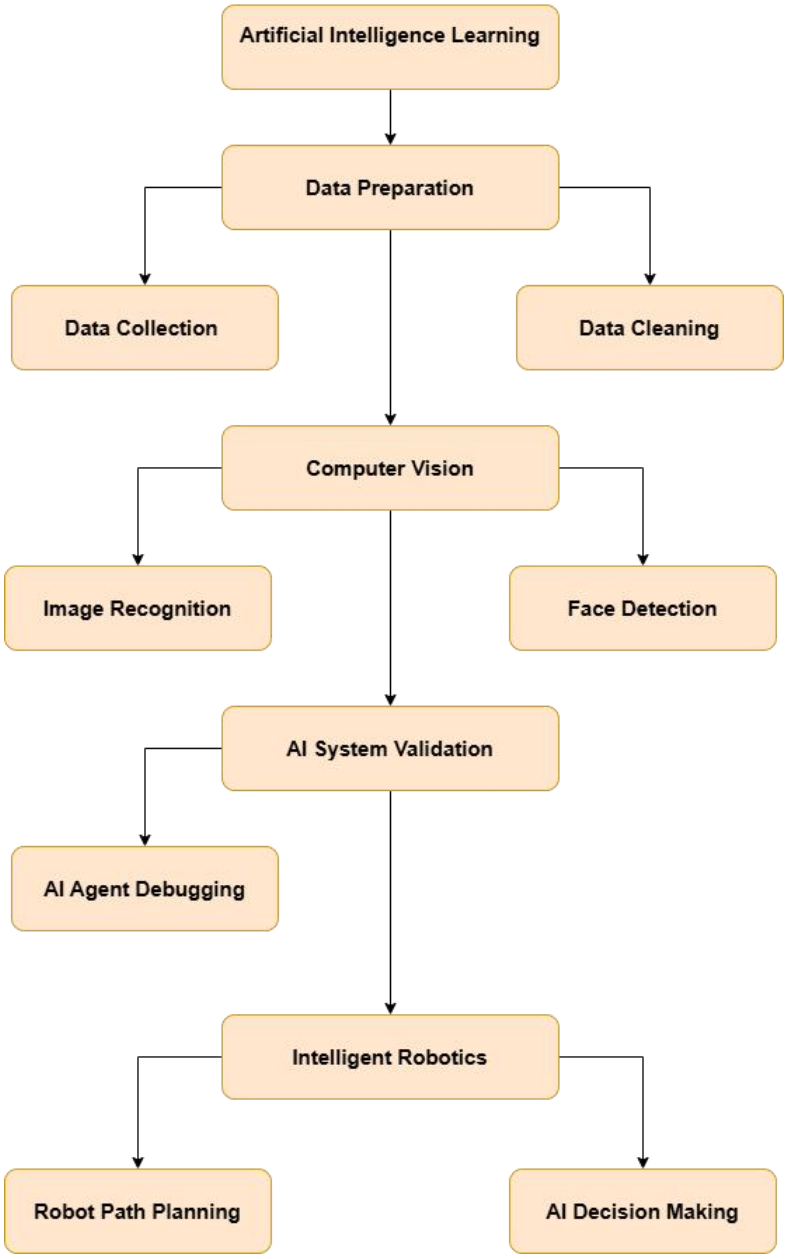


Figure 2.2: AI Learning Architecture

A tree diagram tracing the AI learning pipeline from data preparation through computer vision, AI system validation, and intelligent robotics.

Figure 2.2 presents the educational Artificial Intelligence learning pipeline implemented by the application. Instead of teaching programming concepts directly, the platform introduces children to the main stages involved in developing intelligent systems. Learners begin by understanding how data is collected and prepared before discovering computer vision concepts such as image recognition and face detection. They then explore AI system debugging and finally learn how intelligent robots plan paths and make decisions to accomplish specific tasks.

Table 2.1: AI Learning Mapping Across Domains, Concepts, and Mini-Games

AI Domain	Concept Taught	Implemented Mini-Game
Data Preparation	Data Collection	Data Gathering
Data Preparation	Data Cleaning	Data Cleaning
Computer Vision	Image Recognition	Image Recognition Evaluation
Computer Vision	Face Detection	Face Detection
AI Validation	AI System Debugging	AI Agent Debugging
Intelligent Robotics	Robot Path Planning	Robot Path Planning
Intelligent Robotics	AI Decision Making	AI Decision Making

Cross-references each AI domain with the specific concept and mini-game that teaches it.

2.2 Technologies Used

The development of the educational game required the use of several software technologies and development tools. Each technology was selected according to its role in the design, implementation, version control, project management, and communication throughout the internship.

2.2.1 Unity

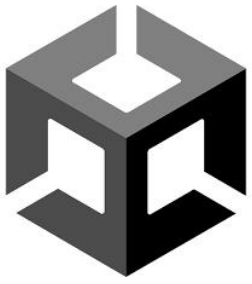


Figure 2.3: Unity Logo

The official Unity logo, representing the game engine used to build the application.

Unity is the main game engine used to develop the application. It provides a complete environment for creating 2D games, managing scenes, animations, user interfaces, physics, audio, and game objects. Unity was also used to implement the seven educational mini-games, the world map, player progression, and scene management.

2.2.2 C#



Figure 2.4: C# Logo

The official C# logo, representing the programming language used throughout the project.

C# is the programming language used throughout the project. It was used to implement the gameplay mechanics, drag-and-drop interactions, level progression, scoring system, tutorials, robot behaviors, and communication between the different game components. Its object-oriented approach allowed the application to be organized into reusable and maintainable scripts.

2.2.3 Visual Studio



Figure 2.5: Visual Studio Logo

The official Visual Studio logo, representing the IDE used to write and debug the code.

Visual Studio served as the Integrated Development Environment (IDE) for the project. It provided advanced code editing, debugging, error detection, and testing tools, facilitating the implementation and maintenance of the application's source code.

2.2.4 Krita



Figure 2.6: Krita Logo

The official Krita logo, representing the digital painting tool used to create the game's graphical assets.

Krita was used to design and edit various graphical assets integrated into the game. It was mainly employed for creating backgrounds, educational illustrations, characters, decorative elements, and visual resources adapted to young learners.

2.2.5 Git



Figure 2.7: Git Logo

The official Git logo, representing the version control system used to track the source code.

Git is a distributed version control system used to track changes made to the project's source code. It enabled efficient version management while preserving the complete development history of the application.

2.2.6 GitHub

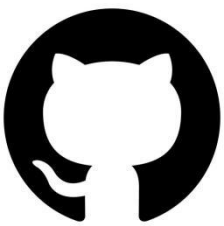


Figure 2.8: GitHub Logo

The official GitHub logo, representing the platform used to host the project's repository.

GitHub was used as the remote repository hosting platform. It allowed project versions to be synchronized, source code to be securely stored, and collaboration with supervisors and team members to be simplified throughout the development process.

2.2.7 ClickUp



Figure 2.9: ClickUp Logo

The official ClickUp logo, representing the tool used to plan and track development tasks.

ClickUp is a project management platform used to organize development tasks, monitor project progress, assign activities, and follow the completion of planned objectives during the internship.

2.2.8 Slack



Figure 2.10: Slack Logo

The official Slack logo, representing the platform used for team communication during the internship.

Slack was used as the primary communication platform between team members and supervisors. It facilitated discussions, project updates, technical support, and the sharing of documents and development progress during the internship.

2.3 Project Management Methodology

The development of the educational game followed the Agile Scrum methodology. Scrum was selected because it supports iterative and incremental development, allowing the project to evolve through continuous implementation, testing, and

feedback. This approach facilitated the integration of new functionalities while ensuring that each completed feature met the project's educational objectives.

2.3.1 Scrum Methodology

Scrum is an Agile project management framework that divides a project into short development cycles called Sprints. At the end of each Sprint, a functional increment of the product is delivered, tested, and reviewed before starting the next iteration. This iterative process improves flexibility, promotes collaboration, and enables rapid adaptation to changing requirements.

2.3.2 Scrum Roles

The Scrum framework defines three main roles that contribute to the successful development of the project.

Product Owner

The Product Owner defines the project requirements, prioritizes the Product Backlog, and ensures that the developed features satisfy the educational objectives of the application.

Scrum Master

The Scrum Master facilitates the development process by ensuring that Scrum practices are correctly applied, removing obstacles encountered during development, and promoting effective collaboration within the team.

Development Team

The Development Team is responsible for designing, implementing, testing, and integrating the different components of the educational game. Throughout the internship, development activities included gameplay programming, user interface implementation, debugging, asset integration, and feature optimization.

Table 2.2: Scrum Roles and Responsibilities

Scrum Role	Team Member	Responsibilities
Product Owner	Mouadh Mkadmi	Defines the project vision, prioritizes the product backlog, validates completed features, and ensures that the developed application meets the educational objectives and project requirements.
Scrum Master	Yassine Alaoui	Supervises the development process, provides technical guidance, removes development obstacles, and ensures that Scrum practices are properly followed throughout the project.
Development Team	Ranym Moussa	Designs, develops, tests, and integrates the educational mini-games, implements the user interface and gameplay mechanics, and manages project updates using the Scrum workflow.

Lists the three Scrum roles held during the project, who filled each one, and their responsibilities.

The project was developed following the Scrum framework. Each team member assumed a specific role to ensure efficient project management, clear communication, and continuous delivery of functional increments. Table 2.2 summarizes the Scrum roles adopted throughout the development process.

2.3.3 Scrum Artifacts

Scrum defines three main artifacts that organize and monitor project development.

Product Backlog

The Product Backlog contains all functional and technical requirements identified for the educational application. Each backlog item represents a feature or improvement that contributes to the final product.

Sprint Backlog

The Sprint Backlog contains the set of tasks selected from the Product Backlog for implementation during the current Sprint. These tasks are planned according to their priority and estimated development effort.

Product Increment

The Product Increment represents the functional result obtained at the end of each Sprint. Each increment is tested and validated before being integrated into the complete educational application.

2.3.4 Scrum Events

The Scrum methodology is organized around several recurring events that structure the development process.

Sprint Planning

Sprint Planning defines the objectives and tasks to be completed during the upcoming Sprint. Development priorities are established according to the Product Backlog.

Daily Scrum

Daily Scrum meetings allow the development team to monitor project progress, discuss completed tasks, identify encountered issues, and plan the activities for the following day.

Sprint Review

At the end of each Sprint, the implemented functionalities are demonstrated and evaluated. Feedback collected during this phase is used to improve future iterations.

Sprint Retrospective

The Sprint Retrospective is conducted after each Sprint to analyze the development process, identify strengths and weaknesses, and define improvements for the next iteration.

2.3.5 Application of Scrum to the Project

The Scrum methodology was applied throughout the development of the educational game. Development activities were organized using ClickUp, where tasks were planned, assigned, and monitored according to their priorities. Team communication and progress discussions were carried out through Slack, enabling efficient collaboration with the internship supervisor and development team.

Each completed feature was integrated, tested, and validated before continuing with the implementation of new functionalities. This iterative workflow improved software quality, simplified debugging, and ensured continuous progress throughout the internship.

2.4 Product Backlog

The Product Backlog defines the functional requirements of the educational AI game in the form of user stories. Each user story describes a feature from the learner's perspective and represents a value that the application must deliver. The backlog was continuously refined throughout the development process according to the project's priorities and educational objectives.

Table 2.3: Product Backlog

Sprint	User Story ID	User Story	Priority
Sprint 1	US1	As a learner, I want to sort different animals into their correct categories so that I can understand the concept of data gathering.	High
	US2	As a learner, I want to receive immediate feedback after each classification so that I can identify my mistakes.	High
	US3	As a learner, I want to complete the activity and automatically return to the Game Map.	Medium
Sprint 2	US4	As a learner, I want to classify different objects into their appropriate categories to understand data cleaning.	High
	US5	As a learner, I want to progress through several levels and sub-levels with increasing difficulty.	High
	US6	As a learner, I want my progress to be saved after completing each level.	Medium
Sprint 3	US7	As a learner, I want to evaluate the robot's image recognition predictions.	High

Sprint	User Story ID	User Story	Priority
	US8	As a learner, I want to receive immediate feedback after every evaluation.	High
	US9	As a learner, I want to unlock the next educational activity after completing the game.	Medium
Sprint 4	US10	As a learner, I want to determine whether the robot correctly detects faces in images.	High
	US11	As a learner, I want to complete all evaluation rounds before returning to the Game Map.	Medium
Sprint 5	US12	As a learner, I want to identify faulty AI agents and remove them while preserving the correct ones.	High
	US13	As a learner, I want to receive immediate visual feedback for each action.	Medium
Sprint 6	US14	As a learner, I want to arrange movement commands to guide the robot safely through a maze.	High
	US15	As a learner, I want the robot to execute my commands and display the result.	High
Sprint 7	US16	As a learner, I want to arrange action cards that help the robot overcome obstacles and reach its destination.	High
	US17	As a learner, I want to understand how AI selects the appropriate action for different situations.	High
Sprint 8	US18	As a learner, I want to access all unlocked educational activities through the Game Map.	High
	US19	As a learner, I want my progression to be automatically saved and displayed on the map.	High

Lists every user story in the product backlog, grouped by sprint, with its priority.

2.5 Sprint Planning

Sprint Planning was conducted before the beginning of each development iteration to define the sprint objective, select the corresponding user stories from the Product Backlog, and estimate the implementation duration. The project was completed over a period of four months (16 weeks), during which each sprint delivered a functional increment of the educational application.

Table 2.4: Sprint Planning

Sprint	Sprint Goal	Selected User Stories	Duration
Sprint 1	Develop the **Data Gathering** educational activity.	US1 – US3	Weeks 1–2
Sprint 2	Develop the **Data Cleaning** educational activity.	US4 – US6	Weeks 3–4
Sprint 3	Develop the **Image Recognition Evaluation** educational activity.	US7 – US9	Weeks 5–6
Sprint 4	Develop the **Face Detection** educational activity.	US10 – US11	Weeks 7–8
Sprint 5	Develop the **AI Agent Debugging** educational activity.	US12 – US13	Weeks 9–10
Sprint 6	Develop the **Robot Path Planning** educational activity.	US14 – US15	Weeks 11–12
Sprint 7	Develop the **AI Decision Making** educational activity.	US16 – US17	Weeks 13–14
Sprint 8	Develop the **Game Map** and integrate the complete application.	US18 – US19	Weeks 15–16

Lists each sprint's goal, the user stories it covers, and its two-week duration across the sixteen-week project.

2.6 Artificial Intelligence Module

2.6.1 Introduction

To further enrich the educational experience, an Artificial Intelligence module has been incorporated into the application. Unlike the previous activities, which introduce AI concepts through interactive simulations, this module integrates a real speech recognition system that enables learners to control a virtual robot using voice commands.

The primary objective of this module is to provide children with a practical understanding of how intelligent systems perceive and interpret spoken language. By interacting directly with the AI model, learners gain hands-on exposure to one of the most widely adopted applications of Artificial Intelligence in modern technology.

At the current stage of development, the system recognizes six spoken categories: four navigation commands (up, down, left, right) together with two rejection categories, unknown and silence, which allow the model to withhold a prediction when no clear command is spoken. The integration of Arabic and French speech

recognition is planned as part of the future evolution of the application, with the objective of offering a multilingual educational experience.

2.6.2 Artificial Intelligence Domain

Artificial Intelligence (AI) is a multidisciplinary field of computer science dedicated to the design of systems capable of performing tasks that traditionally require human cognitive abilities, including learning, reasoning, perception, and decision-making.

The proposed solution belongs to the domain of Automatic Speech Recognition (ASR), commonly referred to as Speech Recognition. This technology enables computer systems to convert spoken language into machine-interpretable commands, allowing natural interaction between users and intelligent applications.

From a machine learning perspective, the developed solution is based on Supervised Learning, where the model is trained using labeled audio recordings. During the learning process, the network establishes the relationship between speech patterns and their corresponding navigation commands, enabling accurate prediction when processing previously unseen voice inputs.

The technological hierarchy of the adopted solution is presented in Figure 2.11.

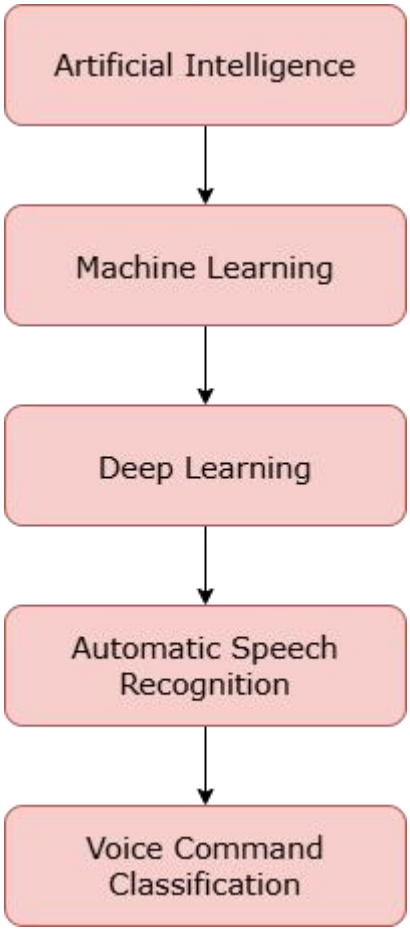


Figure 2.11. Artificial Intelligence hierarchy adopted in the proposed solution.

A flowchart placing the voice recognition feature within the broader AI field, narrowing from Artificial Intelligence down to Voice Command Classification.

2.6.3 Deep Learning Model

The speech recognition system was developed using a Convolutional Neural Network (CNN) implemented with the PyTorch deep learning framework.

The proposed architecture, referred to as VoiceCNN, is designed to automatically extract meaningful acoustic features from speech signals and classify them into one of the six predefined categories. The network takes as input a single-channel log-mel spectrogram of shape 98×40 (98 time frames by 40 mel-frequency bands) and outputs a score for each of the six classes. It consists of multiple convolutional layers responsible for feature extraction, followed by fully connected layers that perform the final classification task. ReLU activation functions are employed to improve the network's learning capability, while Dropout regularization minimizes overfitting and enhances generalization performance.

The overall processing architecture is illustrated in Figure 2.12.

Figure 2.12. VoiceCNN processing architecture.

2.6.4 Model Selection

Several deep learning architectures have been successfully applied to speech recognition problems, including Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, Transformer-based architectures, and Convolutional Neural Networks (CNNs).

For the proposed educational application, the CNN architecture was selected due to its favorable balance between recognition accuracy, computational efficiency, and execution speed. Unlike sequential models, CNNs effectively capture local spectral characteristics of speech while requiring significantly lower computational resources, making them particularly suitable for real-time educational applications developed with Unity.

Furthermore, the lightweight nature of the architecture facilitates deployment on standard computing devices without requiring specialized hardware.

Table 2.5. Justification for Selecting CNN

Evaluation Criterion	CNN
Speech classification performance	Excellent
Computational efficiency	High
Real-time inference capability	Excellent
Integration with Unity	Excellent
Suitability for compact datasets	High

Compares the CNN architecture against the evaluation criteria used to justify its selection for the speech recognition module.

2.6.5 Dataset Preparation

The training dataset was constructed using the Google Speech Commands v0.02 dataset, a publicly available collection containing thousands of labeled recordings of isolated English words produced by numerous speakers.

Within the current implementation, the classifier distinguishes six categories: the four navigation commands (up, down, left, right) required by the educational activity, together with an unknown class covering out-of-vocabulary words and a silence class covering background noise or the absence of speech. Combined with a

confidence threshold applied at inference time, this rejection mechanism prevents the robot from reacting to unclear or accidental audio input.

Future versions of the application will extend the dataset by incorporating Arabic and French voice recordings, thereby enabling multilingual interaction and improving the accessibility of the educational platform.

Table 2.6. Voice Command Categories

Command	Function
Up	Moves the robot upward
Down	Moves the robot downward
Left	Moves the robot left
Right	Moves the robot right
Unknown	Represents unsupported spoken words
Silence	Represents the absence of speech

Lists the six voice command categories recognized by the model and the movement each one triggers.

2.6.6 Audio Preprocessing

Prior to training, each audio recording undergoes a sequence of preprocessing operations intended to standardize the input data and enhance the learning performance of the neural network.

Each one-second recording, sampled at 16 kHz (16,000 raw audio samples), is first normalized to reduce amplitude variations. The signal is then divided into overlapping analysis frames of 400 samples (25 ms) with a hop size of 160 samples (10 ms), producing 98 frames per clip. Each frame is windowed with a Hann function and converted to the frequency domain using a 512-point Fast Fourier Transform. The resulting power spectrum is projected onto 40 triangular mel filters spanning 20 Hz to 8 kHz, and the filter outputs are log-compressed to obtain the final log-mel spectrogram, a 98×40 matrix that serves as the input to VoiceCNN.

This exact pipeline, including the sample rate, frame length, hop size, FFT size, number of mel bands, and frequency range, is re-implemented independently in C# (MelSpectrogram.cs) so that Unity Sentis reproduces the same features at inference time as those used during training in PyTorch. Since Sentis performs fully offline inference with no access to Python audio libraries, this parity between the training and runtime pipelines is essential for the model's predictions to remain valid once deployed in the game.

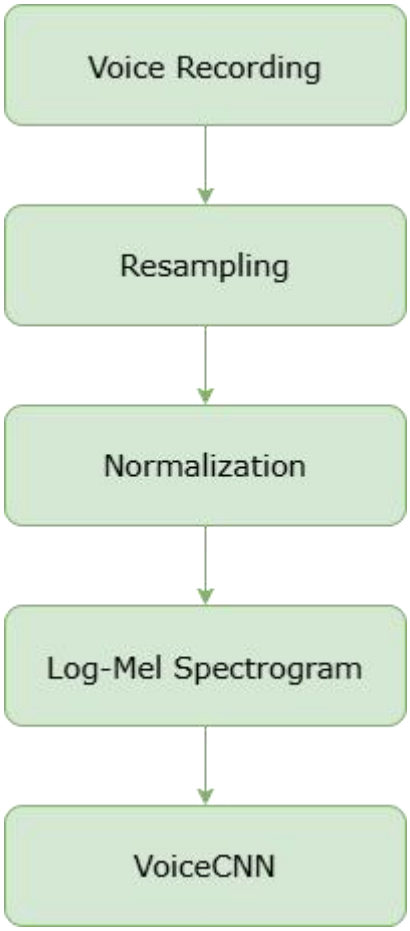


Figure 2.13. Audio preprocessing pipeline.

A flowchart of the steps applied to each recording before classification: voice recording, resampling, normalization, conversion to a log-mel spectrogram, and input into VoiceCNN.

2.6.7 Model Training

The neural network was trained using the PyTorch deep learning framework. During training, the dataset was divided into training and validation subsets to evaluate the generalization capability of the model throughout the learning process.

The optimization procedure relies on the Adam optimizer, while the Cross-Entropy Loss function is employed to quantify the classification error between predicted and expected outputs. Throughout successive training epochs, the network parameters are iteratively updated until satisfactory recognition performance is achieved.

The model exhibiting the highest validation accuracy is retained for deployment within the educational application.

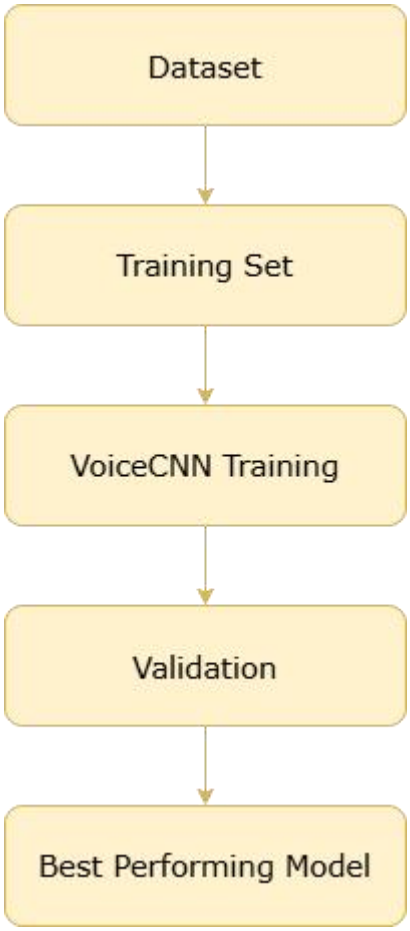


Figure 2.14. Model training workflow.

A flowchart of the training process: preparing the dataset, splitting it into a training set, training VoiceCNN, validating its performance, and retaining the best performing model.

2.6.8 Integration with the Educational Application

Following the training phase, the neural network is exported in the ONNX format to ensure interoperability with the Unity game engine. The exported model is loaded through Unity Sentis using ModelLoader.Load and executed on the CPU backend, enabling fully offline inference directly within the educational application without any dependency on network connectivity or specialized graphics hardware.

During gameplay, the learner presses and holds a microphone button to record up to three seconds of audio; the recording stops as soon as the button is released. The captured waveform is centered or padded to the exact one-second window expected by the model and converted into a log-mel spectrogram using the same preprocessing routine described above. Sentis returns raw class scores, which are converted into probabilities with a softmax function computed manually in C#. If the

highest probability falls below a confidence threshold of 60%, or corresponds to the silence or unknown class, the prediction is discarded and no action is taken. Otherwise, the recognized command is mapped to the corresponding movement block, which is automatically added to the robot's program panel exactly as if the learner had dragged it manually, allowing voice and drag-and-drop input to be used interchangeably during the Robot Path Planning activity.

Figure 2.15. Integration of the speech recognition model within the Unity application.

2.6.9 Current Status and Future Enhancements

At the time of writing this report, the speech recognition module is undergoing continuous refinement and evaluation. The current implementation successfully recognizes English navigation commands, demonstrating the feasibility of integrating real-time speech recognition into an educational game environment.

Future development will focus on extending the model to support Arabic and French, expanding the training dataset with recordings from diverse speakers, improving robustness in noisy environments, and optimizing the model to achieve higher recognition accuracy and faster inference performance. These enhancements aim to provide a more inclusive and effective multilingual learning experience while further demonstrating the practical applications of Artificial Intelligence in educational software.

2.7 Child-Centered Design and Software Engineering Practices

Beyond the AI-specific concepts covered in each mini-game, several design and engineering practices are applied consistently across the entire application. These were treated as shared, reusable patterns rather than one-off implementation details, reflecting standard software engineering principles of separation of concerns and code reuse applied to a children's educational context.

2.7.1 Guided Tutorials

Every mini-game is preceded by a dedicated, scripted tutorial rather than a static instructions screen. Each tutorial is implemented as its own component, built as a coroutine-driven sequence of steps rather than a monolithic block of setup code, with a version tailored to each activity: EvaluationTutorialManager for the Image Recognition Evaluation activity, KidTutorialManager for Data Cleaning, and TutorialManager for Data Gathering, among others. At each step, an animated mascot character named Ferid appears next to a right-to-left Arabic speech bubble,

paired with a matching pre-recorded voice line, while a hand icon with a gentle bobbing and squashing animation points directly at whatever the learner is expected to interact with next. A dark overlay, combined with dynamically bringing only the relevant elements above it at each step, reveals the screen piece by piece instead of showing the whole gameplay view at once, which keeps the experience manageable for a five to ten year old audience.

Rather than scripting a fake demonstration, each tutorial drives the real gameplay manager through public callback methods. For instance, the Data Cleaning tutorial deliberately drags a correct animal into the wrong container first, holds a red flash on the target zone while narrating why the placement is wrong, and only then walks through the correct placement with a green flash, before handing control back to the activity manager for free play. This mistake-then-correction sequence is a deliberate pedagogical choice: it lets a child see and understand a wrong answer's consequence in a safe, guided context before ever being scored on one. Because each tutorial calls directly into the same manager classes used during real gameplay instead of keeping a separate simplified copy of the game logic, the guided walkthrough and the real activity can never drift out of sync with one another.

2.7.2 Shared Software Engineering Patterns

Several patterns recur across every mini-game's implementation. A shared component gives every activity manager the same simple way to report that a level is finished, which keeps each mini-game's internal logic separate from how the Game Map interprets and saves progress. Persistent state, including unlocked levels, per-level completion status, and the drone's last position on the map, is centralized in a single progress-tracking singleton built on top of Unity's PlayerPrefs, rather than each activity managing its own save data. Audio feedback for actions such as drops, correct or incorrect verdicts, and level completion is routed through one central audio manager, which keeps sound design consistent across activities that were otherwise built independently of one another.

Arabic text throughout the application, including tutorial dialogue and in-game feedback messages, is rendered through a shared right-to-left text component from the UPersian localization library rather than Unity's default text components, which ensures correct layout everywhere text appears. Finally, round and level progression in every activity is implemented as a Unity coroutine acting as a small state machine, which keeps the sequencing logic, waiting for an animation, then a player action, then the next state, readable as a straightforward series of steps instead of being spread across multiple callback handlers.

Conclusion

This chapter presented the overall architecture of the educational application, the technologies adopted throughout the development process, and the Agile Scrum methodology used for project management. It also introduced the Artificial Intelligence module integrated into the application and described its design principles and development approach. The next chapters focus on the implementation of each educational activity through the different development iterations.

Chapter 3

Sprint 1 - 3

Introduction

This chapter presents the analysis, design, and implementation of the educational application. The development process was divided into several sprints, each corresponding to one educational mini-game. For every sprint, the functional requirements, system design, UML diagrams, and implementation are presented to illustrate the development process followed throughout the project.

3.1 Sprint 1: Data Gathering

3.1.1 Sprint Overview

The objective of this sprint is to develop the first educational activity, introducing children to the concept of Data Gathering, which represents the initial stage of the Artificial Intelligence workflow. The activity demonstrates that AI systems require sufficient and well-organized data before they can perform learning or make accurate predictions.

The gameplay consists of dragging different animal elements into their corresponding categories. Through this interactive activity, learners understand the importance of collecting and organizing data correctly before it can be used to build intelligent systems.

3.1.2 Sprint Backlog

Table 3.1: Sprint 1 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create and integrate animal assets	High
3	Implement the drag-and-drop interaction mechanism	High
4	Develop category drop zones	High
5	Implement answer validation logic	High
6	Display visual feedback for player actions	Medium
7	Manage game completion and progression	High
8	Return the learner to the Game Map	High

Lists the development tasks completed during Sprint 1 and their priority.

3.1.3 Activity Workflow

The Data Gathering activity introduces the learner to the concept of collecting and organizing training data for Artificial Intelligence. At the beginning of the activity, the system initializes the game scene and displays an interactive tutorial explaining the learning objective and the drag-and-drop mechanics. Once the tutorial is completed, the learner can classify the displayed animals into their corresponding categories. Every successful placement immediately updates the target container, while incorrect attempts are ignored until the correct category is selected. The application continuously verifies the number of correctly classified objects. When all required categories are completed, a success message is displayed and the player's progression is saved before returning to the Game Map.

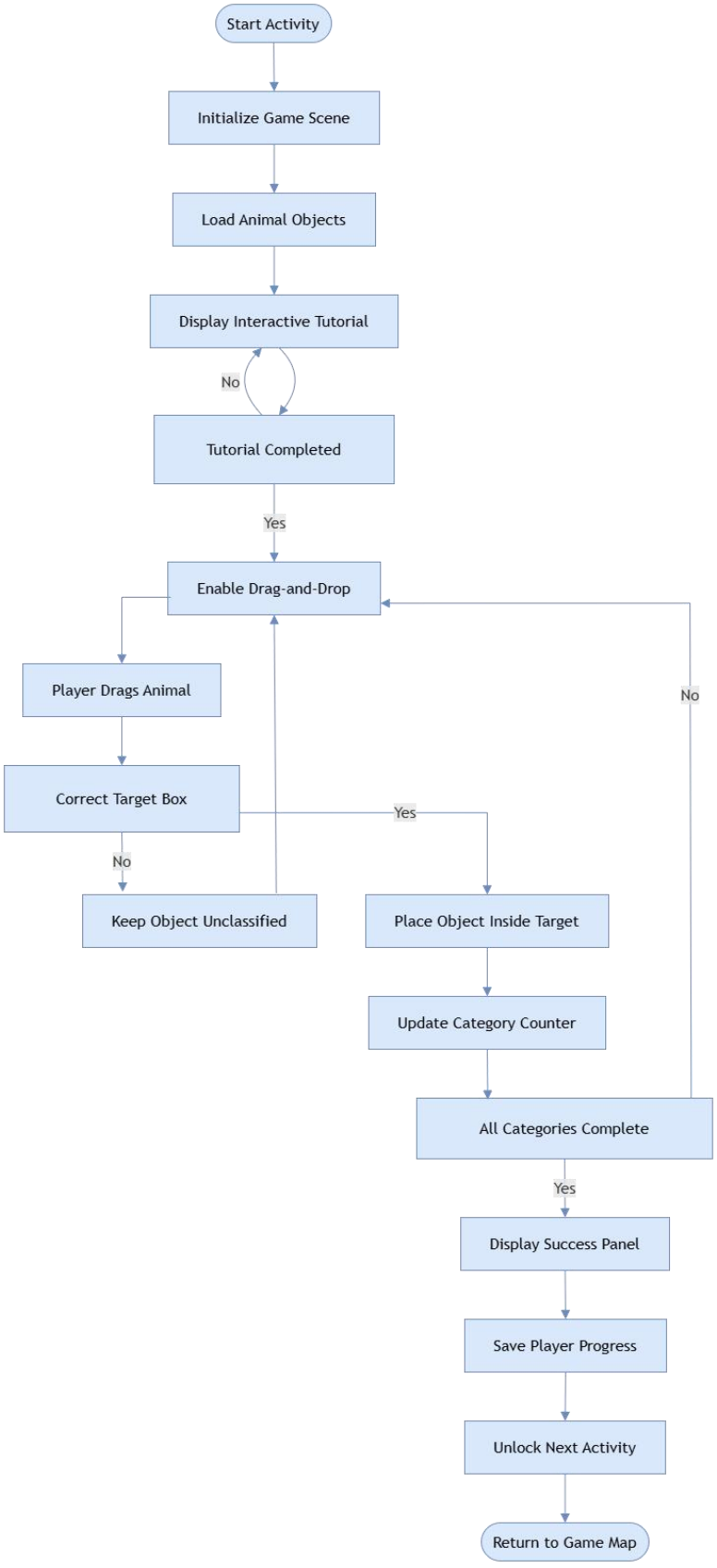

Start Activity
Initialize Game Scene
Load Animal Objects
Display Interactive Tutorial
No
Tutorial Completed
Yes
Enable Drag-and-Drop
Player Drags Animal
Correct Target Box
Yes
No
Keep Object Unclassified
Place Object Inside Target
Update Category Counter
All Categories Complete
No
Yes
Display Success Panel
Save Player Progress
Unlock Next Activity
Return to Game Map

Figure 3.1. Activity Workflow Data Gathering

A flowchart of the Data Gathering activity, from initializing the scene and tutorial through the classification loop to saving progress and returning to the Game Map.

3.1.4 Realization

The Data Gathering activity was implemented using Unity and C#. A drag-and-drop interaction system enables learners to classify animal objects into their appropriate categories. Before the scored activity begins, a dedicated tutorial walks the learner through a mascot-guided introduction, including a staged wrong-then-correct placement demonstration, using the same drag-and-drop components as the real game. Each interaction is validated in real time, and immediate visual feedback is provided to reinforce the learning objective. Once all objects have been correctly classified, the activity is completed, the player's progression is saved, and the application automatically returns to the Game Map.

The following figures illustrate the main interfaces of the Data Gathering activity.



Figure 3.2: Data Gathering gameplay tutorial.
Ferid the mascot introduces the activity, pointing out the animals floating above their glass display pedestals before gameplay begins.



Figure 3.3: Data Gathering gameplay interface

The main play screen, showing several animal bubbles floating above three empty category pedestals waiting to be sorted.



Figure 3.4: Correct object classification

Several animals placed inside their matching pedestals, with one animal bubble still left to be sorted.



Figure 3.5: Success Panel

The level-completion popup congratulating the learner for correctly sorting every animal.

3.2 Sprint 2: Data Cleaning

3.2.1 Sprint Overview

The second sprint focuses on the implementation of the Data Cleaning activity. Its objective is to demonstrate that collected data must be organized and cleaned before being used to train Artificial Intelligence models. This educational activity encourages learners to identify and correctly classify different objects into their appropriate categories.

Unlike the previous activity, this sprint introduces multiple levels and sub-levels with progressively increasing difficulty, allowing learners to strengthen their observation and classification skills.

3.2.2 Sprint Backlog

Table 3.2: Sprint 2 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create educational assets	High
3	Implement drag-and-drop mechanics	High
4	Develop category validation zones	High
5	Implement multi-level and sub-level management	High
6	Validate learner interactions	High

No.	Development Tasks	Priority
7	Display visual feedback and scoring	Medium
8	Update player progression	High

Lists the development tasks completed during Sprint 2 and their priority.

3.2.3 Activity Workflow

The Data Cleaning activity simulates the preprocessing stage commonly performed before training Artificial Intelligence models. After loading the selected level, the application initializes the current sub-level, activates the required number of objects, and randomly distributes them between the two classification zones. To encourage active learning, the system intentionally places a portion of the objects in incorrect categories, preventing the learner from succeeding without interaction. After reviewing the data, the learner validates the arrangement. If every object has been correctly sorted, the system advances to the next sub-level or completes the level. Otherwise, the current round is regenerated with a new randomized distribution, allowing the learner to attempt the activity again.

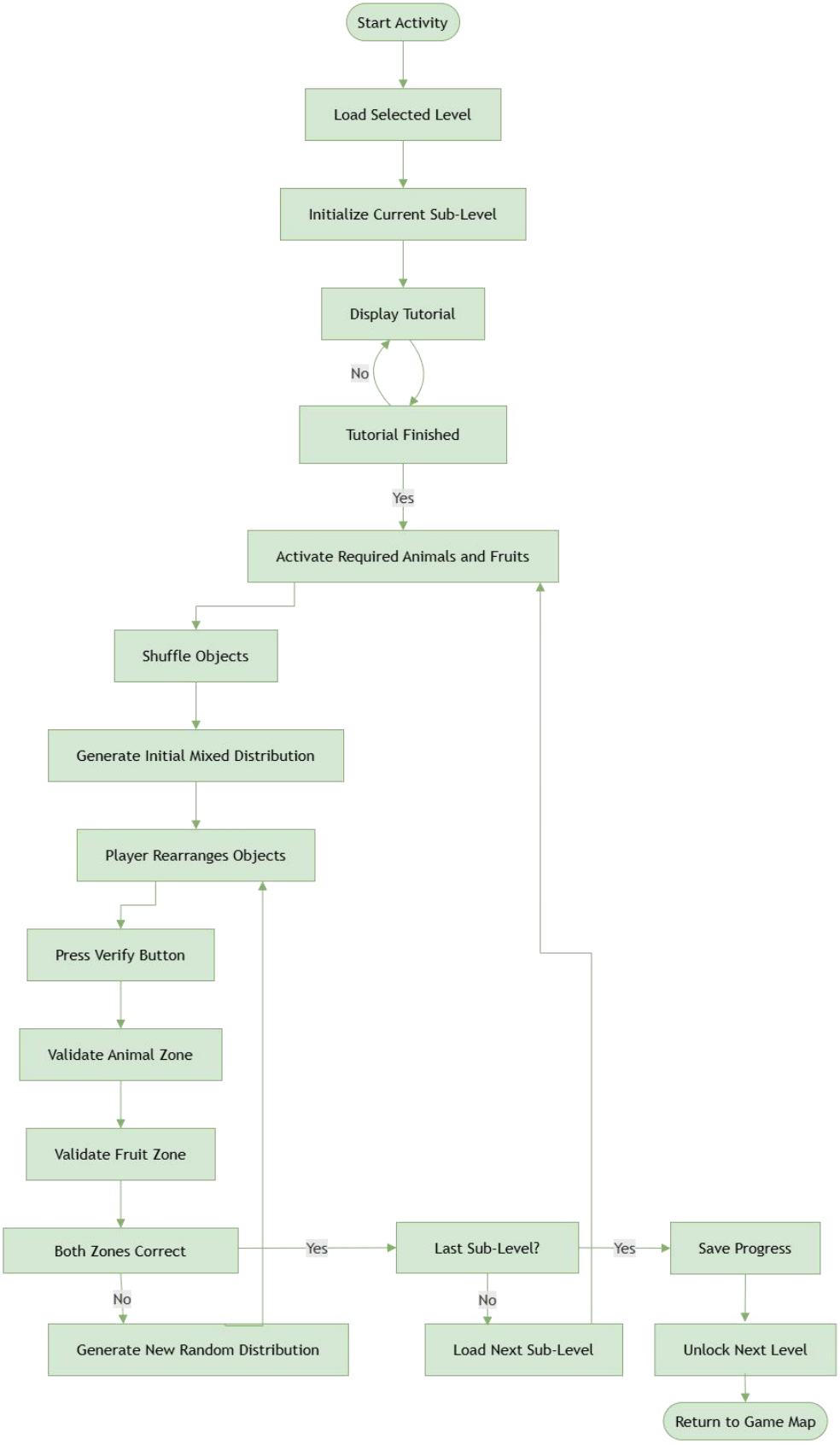

Start Activity
Load Selected Level
Initialize Current Sub-Level
Display Tutorial
No
Tutorial Finished
Yes
Activate Required Animals and Fruits
Shuffle Objects
Generate Initial Mixed Distribution
Player Rearranges Objects
Press Verify Button
Validate Animal Zone
Validate Fruit Zone
Both Zones Correct
Yes
Last Sub-Level?
Yes
Save Progress
No
No
Generate New Random Distribution
Load Next Sub-Level
Unlock Next Level
Return to Game Map

Figure 3.6. Activity Workflow Data cleaning

A flowchart of the Data Cleaning activity, showing the sub-level loop in which objects are deliberately mis-sorted, then reviewed and corrected before the learner submits an answer.

3.2.4 Realization

The activity's difficulty curve is implemented through a two-dimensional structure of levels and sub-levels, where each sub-level is treated as an independent round with its own object count and its own deliberately imperfect starting arrangement. Rather than validating each drag individually, the system checks the whole arrangement only when the learner submits it, which better matches the real data-cleaning workflow of reviewing a full batch before confirming it is clean. A companion guided tutorial runs the learner through the activity beforehand, including an explicit wrong-placement demonstration with narrated red and green flash feedback, before releasing control to the learner for free play. Learners classify different objects into the correct categories while the application validates each interaction and provides immediate feedback. The progression system automatically advances through sub-levels until the current level is completed, after which the learner returns to the Game Map.

The following figures illustrate the main interfaces of the Data Cleaning activity.



Figure 3.7: Data Cleaning gameplay tutorial.
Ferid the mascot walks the learner through the animal and fruit sorting zones before real play begins.



Figure 3.8: Data Cleaning gameplay interface.
The main play screen, showing animals and fruits distributed between the two category zones, some deliberately placed in the wrong one.



Figure 3.9: Wrong classification
An item left in the incorrect zone, highlighted in red to show the learner it needs to be moved.

3.3 Sprint 3: Image Recognition Evaluation

3.3.1 Sprint Overview

The third sprint introduces the concept of Image Recognition, one of the fundamental applications of Computer Vision. Rather than training an AI model, learners evaluate whether the robot correctly identifies the objects displayed on the

screen. This activity highlights the importance of verifying AI predictions to ensure reliable system performance.

The gameplay is organized into multiple rounds, each presenting a new recognition scenario that learners must evaluate before receiving immediate feedback.

3.3.2 Sprint Backlog

Table 3.3: Sprint 3 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create image recognition scenarios	High
3	Display AI predictions	High
4	Implement the prediction evaluation mechanism	High
5	Validate learner responses	High
6	Manage gameplay rounds	High
7	Display visual feedback	Medium
8	Update player progression	High

Lists the development tasks completed during Sprint 3 and their priority.

3.3.3 Activity Workflow

The Image Recognition Evaluation activity allows the learner to assess the predictions generated by an Artificial Intelligence model. After completing the tutorial, the system generates a new evaluation round by assigning labels to the displayed images. Some labels correspond to the correct prediction, while others are intentionally replaced with incorrect values selected from a predefined vocabulary. The robot automatically labels each image before entering the evaluation phase. The learner then analyzes every prediction individually and determines whether the robot's decision is correct or incorrect. After all predictions have been evaluated, the system computes the robot's accuracy, updates the performance indicator, and either starts a new evaluation round or displays the final results when all rounds have been completed.

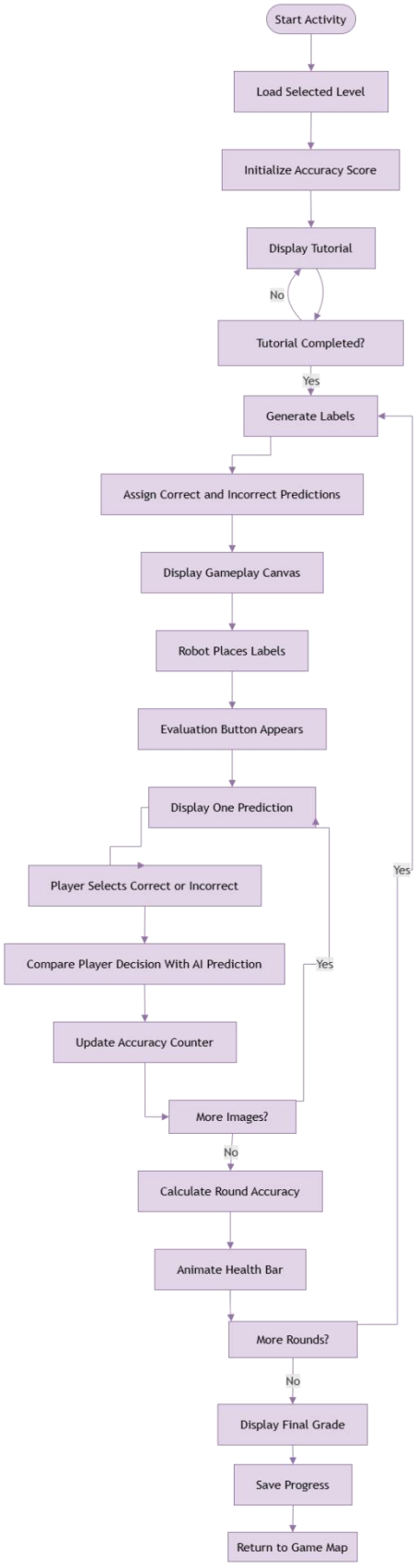

Start Activity
Load Selected Level
Initialize Accuracy Score
Display Tutorial
No
Tutorial Completed?
Yes
Generate Labels
Assign Correct and Incorrect Predictions
Display Gameplay Canvas
Robot Places Labels
Evaluation Button Appears
Display One Prediction
Player Selects Correct or Incorrect
Compare Player Decision With AI Prediction
Yes
Update Accuracy Counter
More Images?
No
Calculate Round Accuracy
Animate Health Bar
More Rounds?
Yes
No
Display Final Grade
Save Progress
Return to Game Map

Figure 3.10. Activity Workflow Image Recognition Evaluation

A flowchart of the Image Recognition Evaluation activity, from the robot labelling the displayed images to the learner judging each prediction and the system updating the robot's accuracy.

3.3.4 Realization

The activity is built around a shared data store that tracks the robot's simulated accuracy, an internal score rather than a real trained model, initialized at 30% and bounded between 10% and 100%, together with the current level and round. It spans three thematic levels (emotions, directions, and animals), each played over four rounds. At the start of a round, the number of images the robot labels correctly is derived directly from the current accuracy value, so a robot with a higher score is visibly more reliable before the learner even starts judging it; the robot then hops between the images and reveals each label in turn. The learner reviews the same labelled images, reused rather than duplicated so the two views stay in sync, and judges each as correct or incorrect. A dedicated guided tutorial walks the learner through one live round before scored play begins, using the same underlying activity manager rather than simulating it separately. If at least half of a round's judgements agree with the ground truth, accuracy increases by a fixed step; otherwise it decreases by the same step, giving a visible, gradually improving health bar. After completing all rounds of a level, a results panel reports the final accuracy and a qualitative grade before the learner returns to the Game Map.

The following figures illustrate the main interfaces of the Image Recognition Evaluation activity.



Figure 3.11: Image Recognition gameplay tutorial.
The robot hops between the photos on the board, placing a label under each one while the learner watches.



Figure 3.12: Image Recognition gameplay interface.
The main play screen, showing the robot's labelled photo board alongside the evaluate button.



Figure 3.13: Evaluation screen.
The second screen, where the learner reviews a single labelled photo and judges it as correct or incorrect.

3.4 UML Modeling

To provide a structural representation of the first three sprints, the following UML diagrams illustrate the common interactions between the player and the educational application, the execution flow of a gameplay session, and the principal software classes involved in the implementation.

3.4.1 Use Case Diagram

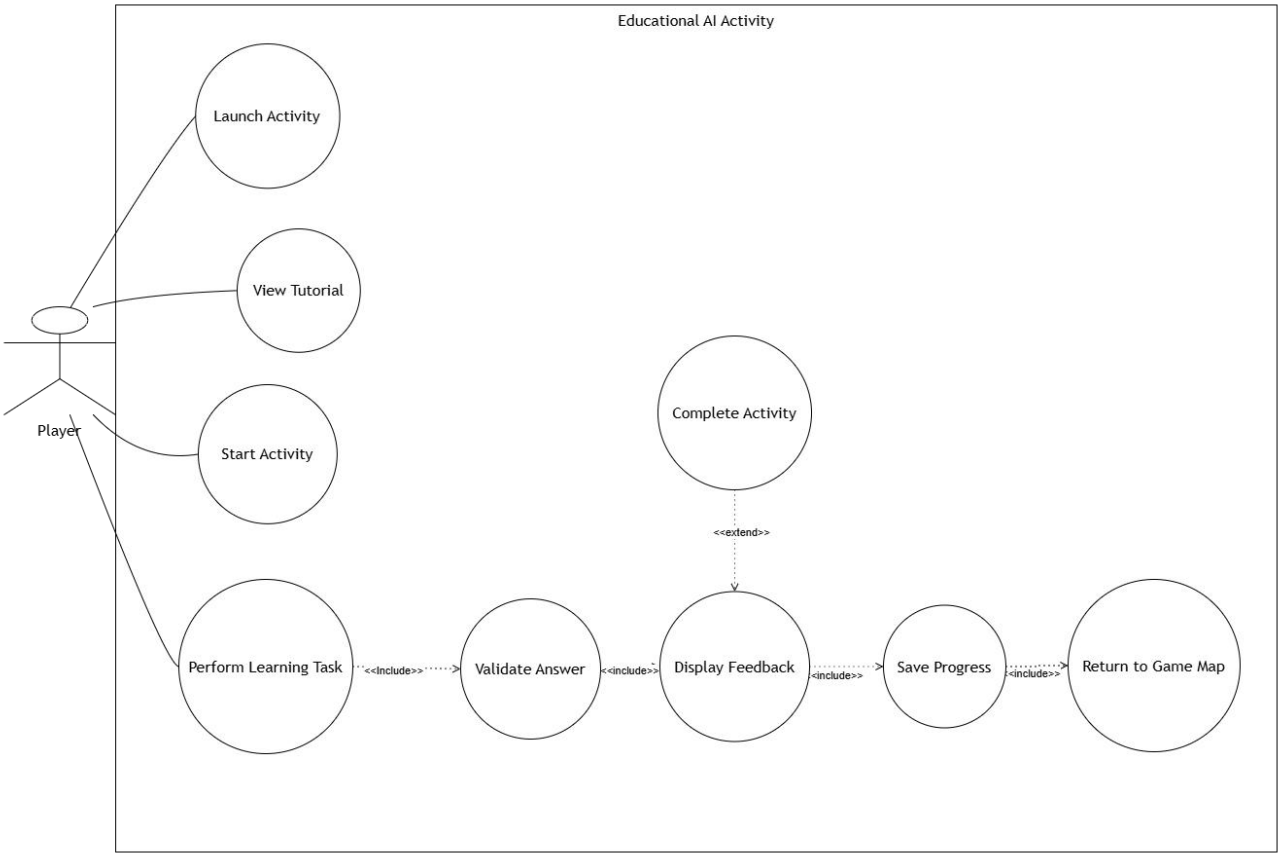


Figure 3.14: Use Case Diagram of the Data Preparation and Image Recognition modules.

A use case diagram showing the player launching an activity, viewing its tutorial, performing the learning task, and receiving feedback before progress is saved.

3.4.2 Sequence Diagram

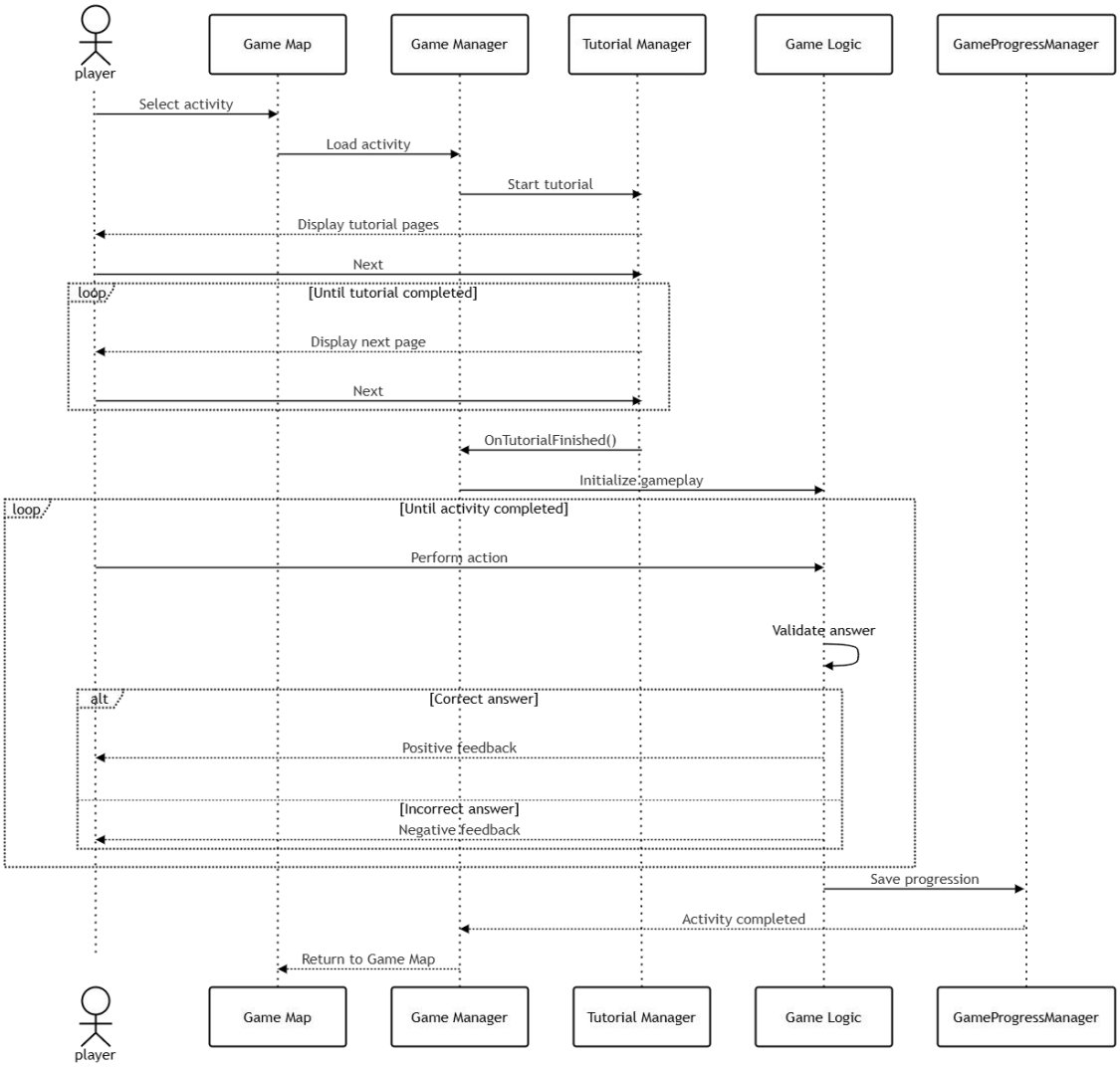


Figure 3.15: Sequence Diagram of the gameplay workflow.

A sequence diagram tracing a full play session, from selecting an activity on the Game Map through the tutorial and gameplay loop to progress being saved and the player returning to the map.

3.4.3 Class Diagram

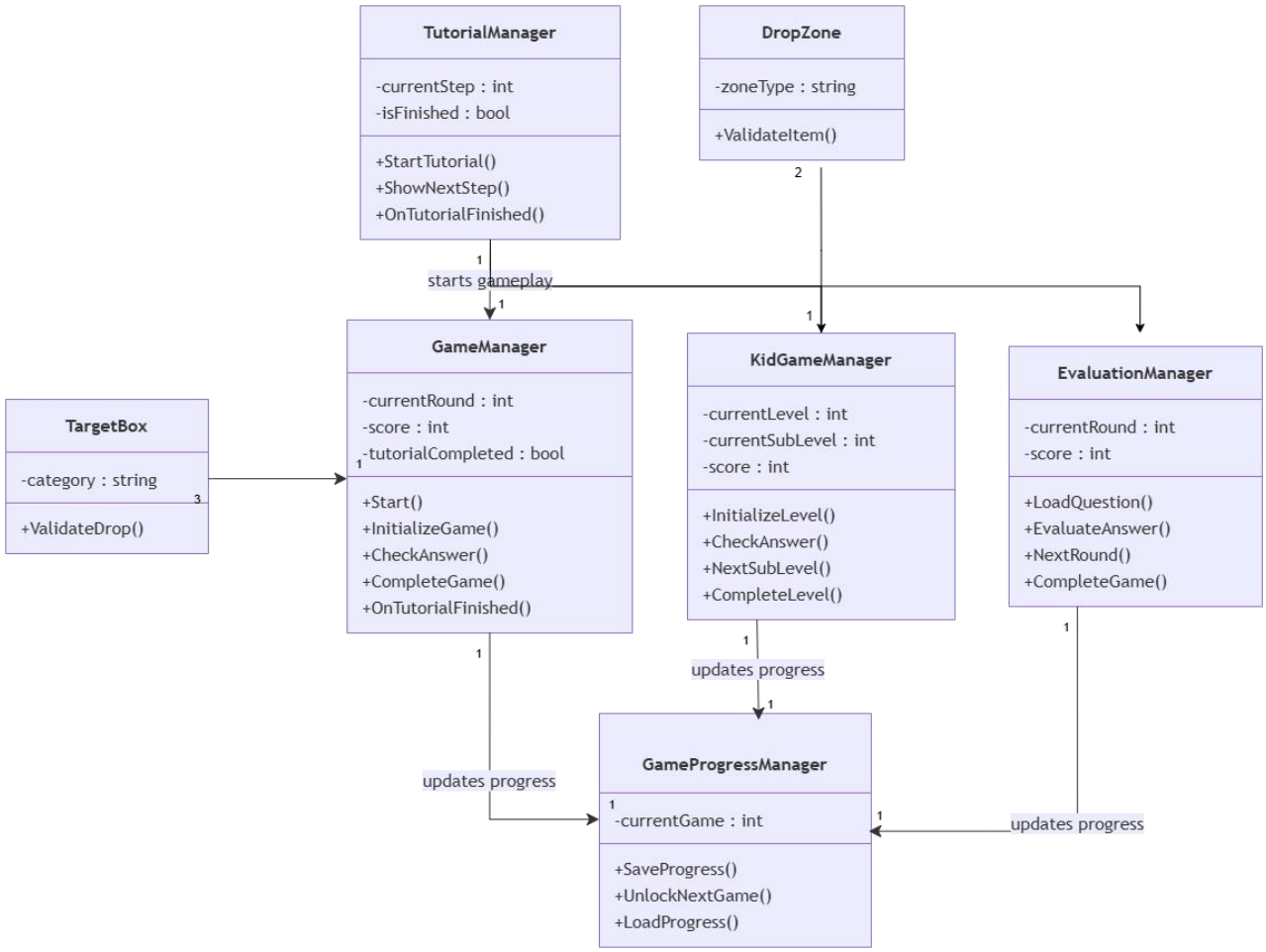


Figure 3.16: Class Diagram of the first three educational modules.

A class diagram showing the relationships between the Tutorial Manager, Drop Zone, Target Box, Game Manager, Kid Game Manager, Evaluation Manager, and Game Progress Manager classes.

Conclusion

This chapter presented the analysis, design, and realization of the first three educational mini-games, covering the concepts of Data Gathering, Data Cleaning, and Image Recognition Evaluation. Together, these modules introduce the initial stages of the Artificial Intelligence learning process through interactive gameplay. The accompanying UML diagrams summarize the common architecture and software interactions implemented within these modules.

Chapter 4

Sprint 4 - 6

Introduction

This chapter presents the analysis, design, and realization of the second group of educational activities developed for the application. These sprints introduce learners to advanced Artificial Intelligence concepts, including Face Detection, AI System Debugging, and Robot Path Planning. Each activity was designed to simplify complex AI concepts through interactive gameplay while maintaining a progressive learning experience. The chapter concludes with the UML diagrams illustrating the common architecture and interactions of these educational modules.

4.1 Sprint 4: Face Detection

4.1.1 Sprint Overview

The objective of this sprint is to introduce learners to the concept of Face Detection, one of the most widely used Computer Vision techniques. The activity demonstrates how Artificial Intelligence systems locate human faces within an image before performing additional recognition or analysis tasks.

During gameplay, the robot places a detection frame on an image containing one or more faces. The learner evaluates whether the detected area is correct and provides feedback accordingly. This interaction helps children understand that AI systems must first detect objects before recognizing or analyzing them.

4.1.2 Sprint Backlog

Table 4.1: Sprint 4 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create face detection scenarios	High
3	Display AI detection results	High
4	Implement the learner evaluation mechanism	High
5	Validate learner responses	High
6	Manage gameplay rounds	High
7	Display visual feedback	Medium
8	Update player progression	High

Lists the development tasks completed during Sprint 4 and their priority.

4.1.3 Activity Workflow

The Face Detection activity introduces the learner to the concept of object localization in Computer Vision. At the beginning of each round, the robot automatically analyzes the displayed image and places a detection square around what it believes to be the face. Depending on its current training score, the robot may place the square correctly, near the face, or far from the target. The learner first evaluates whether the robot's prediction is correct. If the prediction is incorrect, the learner manually adjusts the detection square by dragging it onto the detected face before confirming the correction. The application evaluates the final position of the square, updates the robot's training score, and progressively improves the robot's detection accuracy during subsequent rounds. After all training rounds are completed, the application displays the final result and records the player's progression.

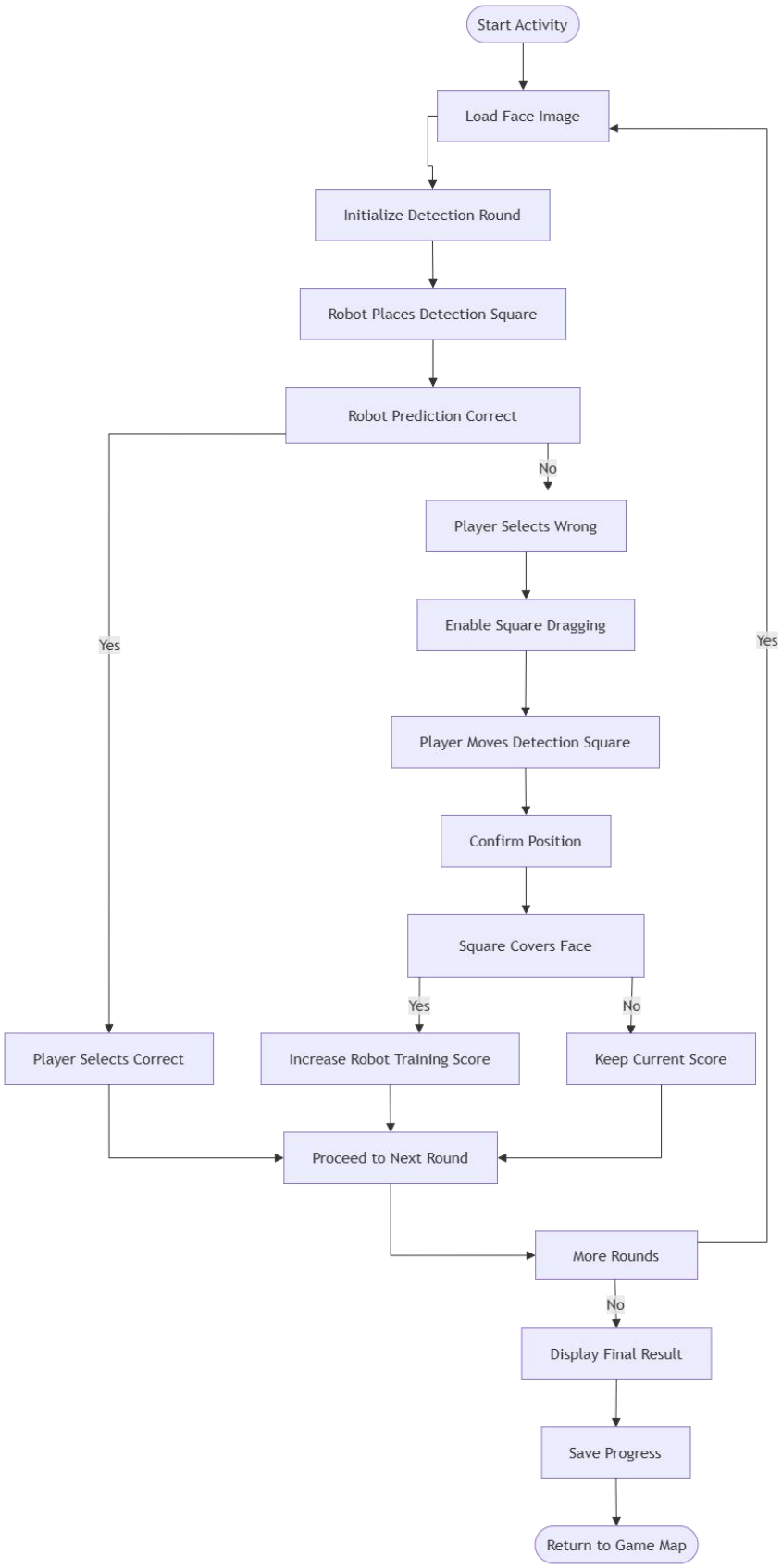

Start Activity
Load Face Image
Initialize Detection Round
Robot Places Detection Square
Robot Prediction Correct
No
Player Selects Wrong
Enable Square Dragging
Yes
Player Moves Detection Square
Confirm Position
Square Covers Face
Yes
No
Player Selects Correct
Increase Robot Training Score
Keep Current Score
Proceed to Next Round
More Rounds
Yes
No
Display Final Result
Save Progress
Return to Game Map

Figure 4.1: Activity Workflow Face Detection

A flowchart of the Face Detection activity, from the robot placing its detection square through the learner's evaluation and correction loop to the final results screen.

4.1.4 Realization

The activity is built around a six-round training loop in which the robot's placement of the detection square is deliberately imprecise at first and improves as an internal training score rises: the first round always places the square far from the face, and later rounds are placed far, near, or exactly on the face depending on score thresholds reached so far. A dishonest verdict, meaning one that disagrees with the true placement, is called out immediately, while a wrong placement the learner correctly catches becomes a hands-on correction step: the learner drags the square onto the face and confirms it, and the training score only increases if the final position lands within a set distance of the true face centre. This progressively improving placement pattern was chosen specifically to give children the experience of watching an AI system get better at a task through repeated correction, rather than presenting a robot that is always right or always wrong. After six rounds the activity records the learner's progress and returns to the Game Map.

The following figures illustrate the main interfaces of the Face Detection activity.



Figure 4.2: Face Detection gameplay interface.
The robot inspecting a photograph, with a highlighted square marking where it believes the face to be.



Figure 4.3: Evaluation example.
An example round in which the learner compares the robot's placed square against the photograph before deciding whether it is correct.



Figure 4.4: Correcting robot's answer.
The learner dragging the detection square onto the correct position after catching a mistaken placement.

4.2 Sprint 5: AI Agent Debugging

4.2.1 Sprint Overview

The objective of this sprint is to introduce the concept of AI System Debugging. Learners discover that Artificial Intelligence systems may contain faulty

components that produce incorrect behavior and therefore need to be identified and corrected.

The activity presents different AI agents, some functioning correctly and others containing errors. Learners must identify the faulty agents and remove them while preserving the correctly functioning ones. This activity develops observation skills while introducing the importance of maintaining reliable AI systems.

4.2.2 Sprint Backlog

Table 4.2: Sprint 5 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create functional and faulty AI agents	High
3	Implement drag-and-drop interactions	High
4	Develop the recycle bin mechanism	High
5	Validate learner actions	High
6	Display visual feedback	Medium
7	Manage gameplay rounds	High
8	Update player progression	High

Lists the development tasks completed during Sprint 5 and their priority.

4.2.3 Activity Workflow

The AI Agent Debugging activity simulates the process of evaluating and correcting an unreliable AI system. At the beginning of each of the six rounds, the application generates a fresh population of agents and randomly labels each one as either well-behaved or faulty, according to the robot's current accuracy: this accuracy starts at 30%, decreases slightly as the rounds progress to keep the challenge consistent, and is always kept between 5% and 95% so that a mix of correct and incorrect agents is always present. The learner inspects the population and drags every faulty agent into a recycle bin while leaving the well-behaved agents untouched. Upon submission, the system compares the learner's choices with the true labels, computing both the proportion of faulty agents correctly removed and the proportion of well-behaved agents removed by mistake, and adjusts the robot's accuracy accordingly. When faulty agents are missed, a guided teaching moment highlights each one in turn before the round is reset, helping the learner recognize the mistake before continuing. After all six rounds are completed, the activity records the learner's progress and returns to the Game Map.

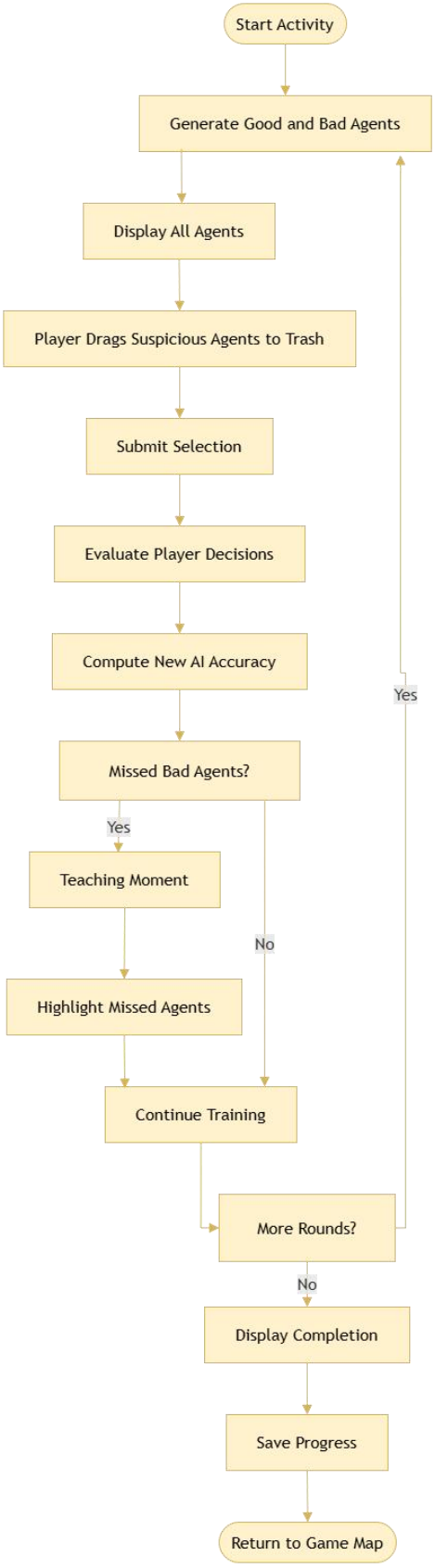

Start Activity
Generate Good and Bad Agents
Display All Agents
Player Drags Suspicious Agents to Trash
Submit Selection
Evaluate Player Decisions
Compute New AI Accuracy
Missed Bad Agents?
Yes
Teaching Moment
No
Highlight Missed Agents
Continue Training
More Rounds?
Yes
No
Display Completion
Save Progress
Return to Game Map

Figure 4.5: Activity Workflow AI Agent Debugging

A flowchart of the AI Agent Debugging activity, showing the round loop in which agents are generated, faulty ones are dragged into the recycle bin, and the robot's accuracy is updated based on the learner's choices.

4.2.4 Realization

The AI Agent Debugging activity was implemented using a drag-and-drop interaction system. Learners identify faulty AI agents and move them into the recycle bin while leaving correctly functioning agents untouched. Quantitatively, each round's accuracy change is computed as 0.20 times the proportion of faulty agents correctly removed, minus 0.15 times the proportion of well-behaved agents removed by mistake. This weights missed faulty agents more heavily than accidental removals, discouraging learners from indiscriminately discarding every agent. Each agent subtly pulses between its base colour and a red or green glow tied to its true label, a soft visual affordance that rewards careful observation without revealing the answer outright, while a shadow placeholder marks its original slot during dragging. Every action is validated immediately to provide real-time feedback and reinforce the learning objective. Once all six rounds are completed successfully, the learner's progression is updated before returning to the Game Map.

The following figures illustrate the main interfaces of the AI Agent Debugging activity.



Figure 4.6: AI Agent Debugging gameplay interface.
A grid of AI agents in varying colours to be sorted, a round tracker across the top, and a trash bin used to discard the faulty ones.



Figure 4.7: Indicate missed agents.
Faulty agents the learner missed are highlighted so they can be recognised before the next round begins.



Figure 4.8: Agent Debugging progress
The round tracker after a submission, showing how many of the six rounds have been completed.

4.3 Sprint 6: Robot Path Planning

4.3.1 Sprint Overview

The objective of this sprint is to introduce the concept of Robot Path Planning. This activity demonstrates that intelligent robots must follow an ordered sequence of movement instructions to safely navigate an environment and reach a predefined destination.

The gameplay is based on a maze composed of obstacles and a target location. Learners arrange directional commands to create a valid navigation path before executing the solution. The robot follows the generated sequence, allowing learners to observe how AI systems plan movements to accomplish specific objectives.

4.3.2 Sprint Backlog

Table 4.3: Sprint 6 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create maze layouts	High
3	Implement robot movement	High
4	Develop directional command cards	High
5	Validate navigation sequences	High
6	Detect collisions with obstacles	High
7	Manage level progression	High
8	Return to the Game Map	High

Lists the development tasks completed during Sprint 6 and their priority.

4.3.3 Activity Workflow

The Robot Path Planning activity guides the learner through a maze that must be solved using directional commands. Four difficulty levels are available, each played over two rounds: the maze layout is procedurally generated using a randomized depth-first search algorithm, while the second round of each level reuses a fixed, hand-designed layout to reinforce the concepts learned in the first. The learner builds a movement program either by dragging directional blocks (up, down, left, right) into the program panel or by holding the microphone button and speaking the corresponding command, recognized by the same speech recognition model presented earlier in this report. Once the Play button is pressed, the robot executes the recorded program step by step, sliding in each direction until it reaches the next wall rather than moving one cell at a time. If a move would cross a wall the run stops immediately and an error message is shown; otherwise the robot continues until the full program has executed. Reaching the highlighted goal cell completes the round and advances to the next round or level, while missing it allows the learner to edit the program and try again.

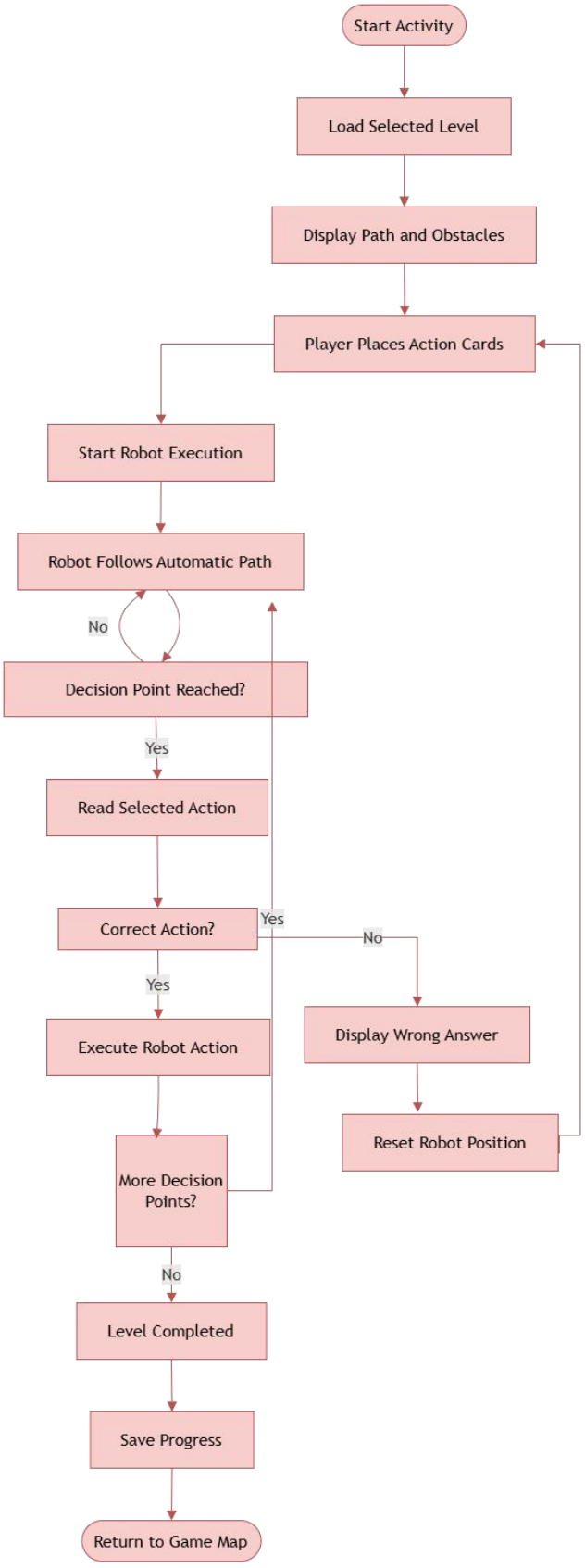

Start Activity
Load Selected Level
Display Path and Obstacles
Player Places Action Cards
Start Robot Execution
Robot Follows Automatic Path
No
Decision Point Reached?
Yes
Read Selected Action
Correct Action?
Yes
No
Yes
Execute Robot Action
Display Wrong Answer
Reset Robot Position
More Decision Points?
No
Level Completed
Save Progress
Return to Game Map

Figure 4.9: Activity Workflow Robot Path Planning

A flowchart of the Robot Path Planning activity, from generating the maze through building and executing the movement program to checking whether the robot reached the goal.

4.3.4 Realization

The Robot Path Planning activity enables learners to construct a sequence of movement instructions that guides the robot through a maze. Movement uses a roll-until-blocked model rather than single-cell steps, a natural fit for a maze built as a grid of walls and open corridors, since it lets a short program of only a few commands cover a full maze without requiring a separate instruction for every individual cell. Across the four levels and two rounds each, eight distinct mazes are presented in total: the first round of every level is procedurally generated so the layout is different each time, while the second round reuses a fixed, hand-designed layout to consolidate what was learned. After executing the generated path, the application validates the solution and immediately displays the result. Successful completion unlocks the next round or level until all levels have been completed, after which the learner returns to the Game Map.

The following figures illustrate the main interfaces of the Robot Path Planning activity.



Figure 4.10: Robot Path Planning gameplay interface.
The maze scene with its obstacles, alongside the set of directional action cards and empty program slots the learner fills in before pressing start.



Figure 4.11: Placing Action Cards

The learner filling the empty program slots with directional action cards before running the sequence.



Figure 4.12: Submit selection

The completed program ready to be executed, just before the learner presses play.

4.4 UML Modeling

The following UML diagrams present the common software architecture shared by the Face Detection, AI Agent Debugging, and Robot Path Planning activities. They illustrate the interactions between the learner and the application, the execution flow during gameplay, and the principal software classes responsible for implementing these educational modules.

4.4.1 Use Case Diagram

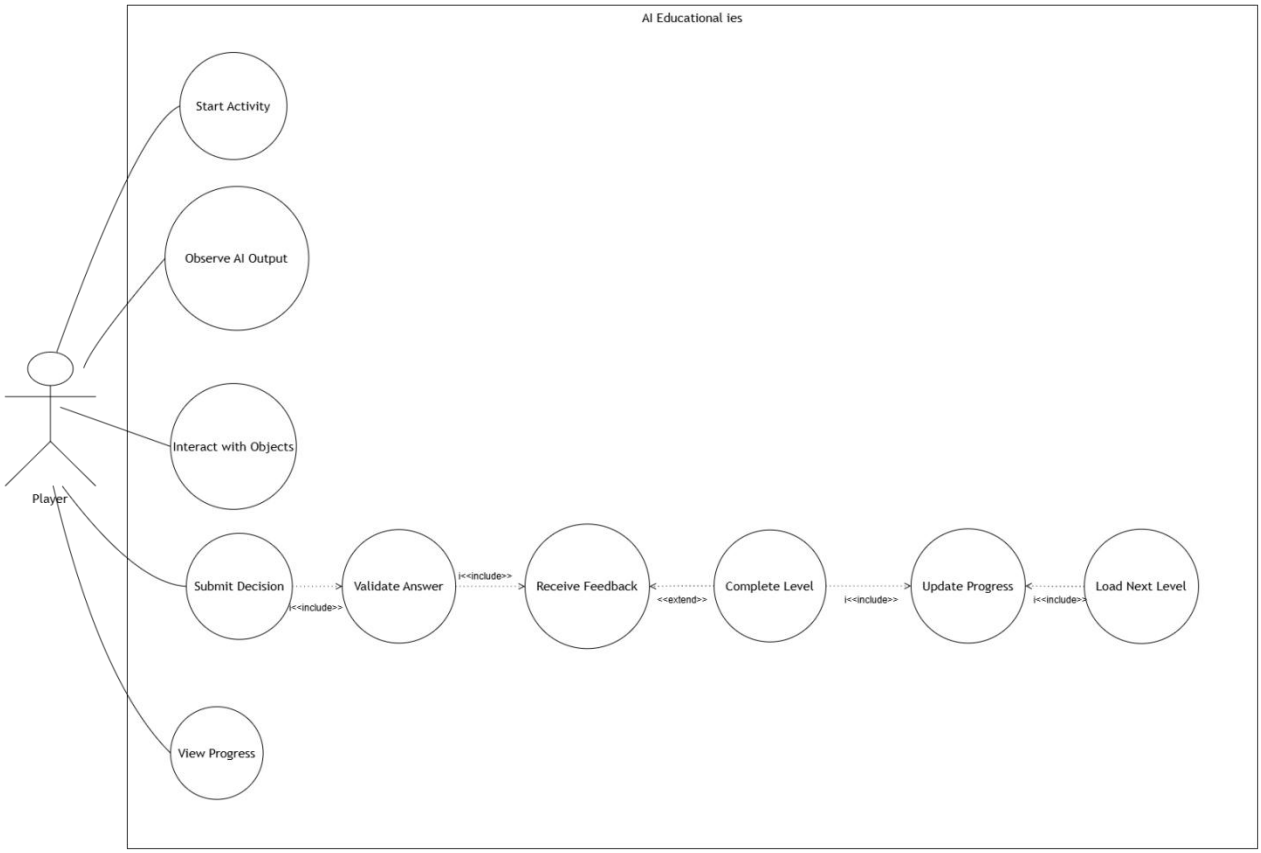


Figure 4.13: Use Case Diagram of the Face Detection, AI Agent Debugging, and Robot Path Planning activities.

The same use case pattern as the earlier chapter, adapted for the Face Detection, AI Agent Debugging, and Robot Path Planning activities.

4.4.2 Sequence Diagram

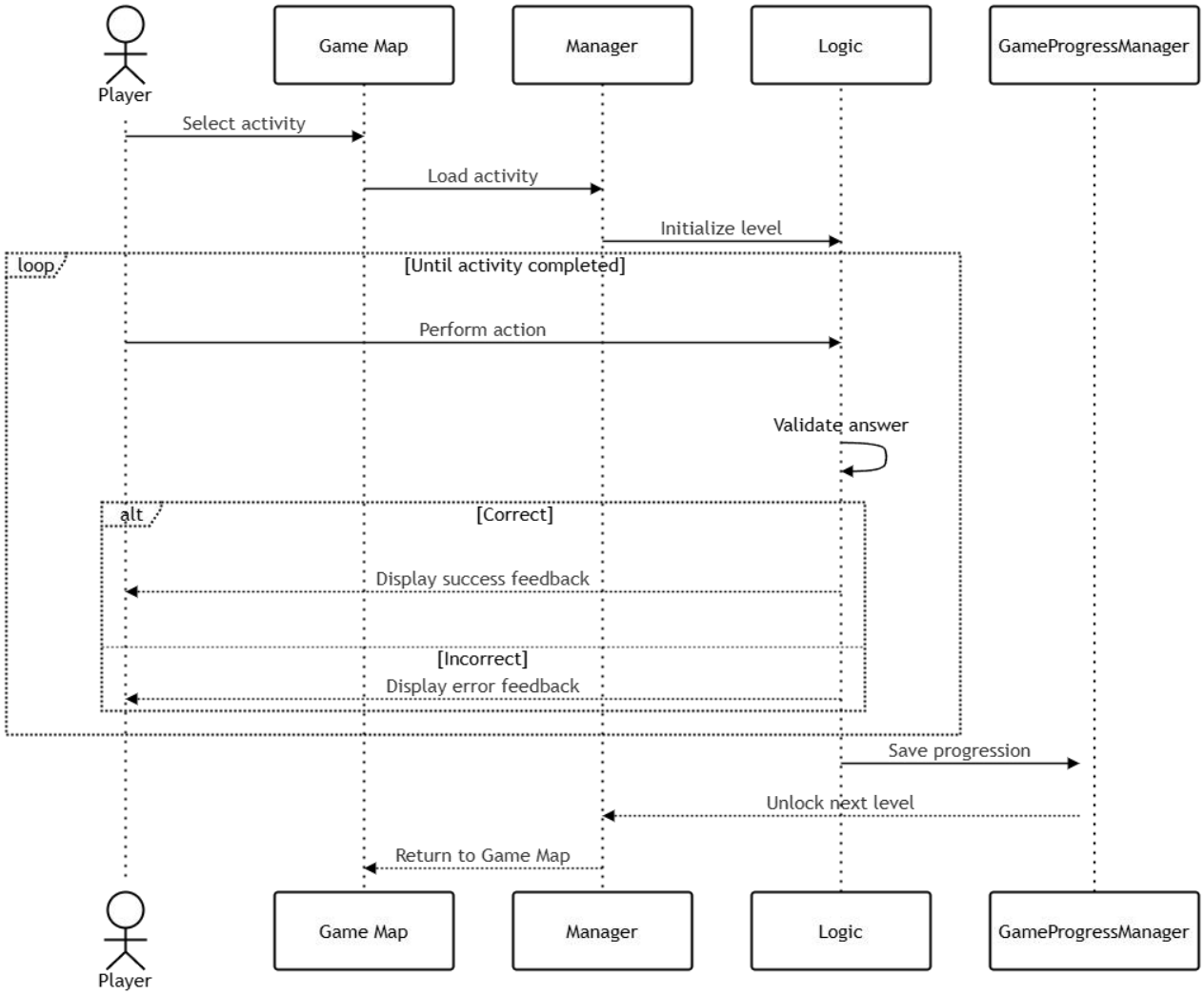


Figure 4.14: Sequence Diagram illustrating the gameplay workflow.

A sequence diagram tracing the shared activity, tutorial, and progress-saving flow used by the Face Detection, AI Agent Debugging, and Robot Path Planning activities.

4.4.3 Class Diagram

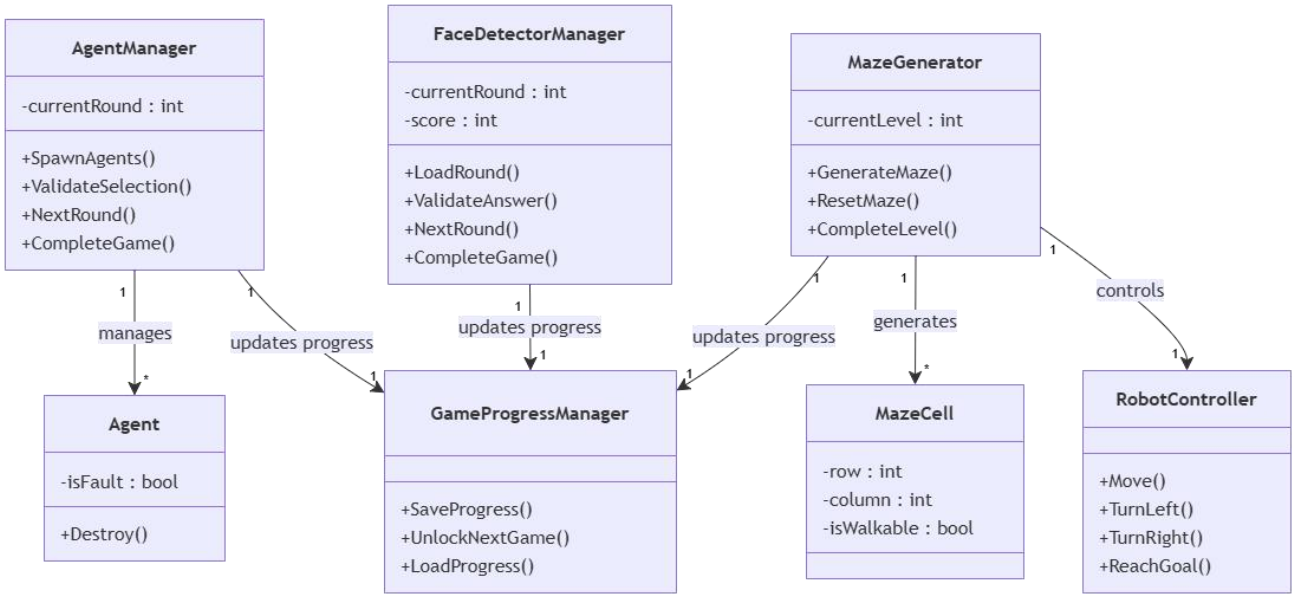


Figure 4.15: Class Diagram of the second group of educational activities.

A class diagram showing the manager and helper classes shared by the second group of activities.

Conclusion

This chapter presented the implementation of three educational activities covering the concepts of Face Detection, AI System Debugging, and Robot Path Planning. Through interactive gameplay, learners discover how AI systems detect human faces, identify faulty system components, and generate navigation paths for autonomous robots. The accompanying UML diagrams summarize the common software architecture and interactions implemented within these modules.

Chapter 5

Sprint 7 - 8

Introduction

This chapter presents the implementation of the final two modules of the educational application. The first sprint introduces the concept of AI Decision Making, allowing learners to understand how intelligent systems select appropriate actions to solve a problem. The second sprint presents the implementation of the Game Map, which serves as the central navigation interface connecting all educational activities while managing player progression and game accessibility. The chapter concludes with the UML diagrams representing the software architecture and interactions of these modules.

5.1 Sprint 7: AI Decision Making

5.1.1 Sprint Overview

The objective of this sprint is to introduce learners to the concept of AI Decision Making. The activity demonstrates that intelligent systems analyze their environment and select the most appropriate action to achieve a specific objective.

The game consists of six progressive levels in which learners arrange action cards representing different robot abilities, such as Jump, Climb, Break Rock, and Use Zipline. After executing the selected sequence, the robot performs each action in order. If the chosen solution is correct, the learner advances to the next level; otherwise, the sequence can be modified and executed again.

5.1.2 Sprint Backlog

Table 5.1: Sprint 7 Backlog

No.	Development Tasks	Priority
1	Design the graphical user interface (GUI)	High
2	Create robot action cards	High
3	Implement drag-and-drop mechanics	High
4	Develop action slots	High
5	Validate the action sequence	High
6	Animate robot actions	High
7	Manage level progression	High
8	Update player progression	High

Lists the development tasks completed during Sprint 7 and their priority.

5.1.3 Activity Workflow

The AI Decision Making activity introduces learners to selecting the correct action for a given situation rather than simply moving forward. Each of the six levels defines a fixed path made up of several segments separated by decision points. The robot automatically travels along each segment, and whenever it reaches a decision point it pauses and checks the action card the learner has placed in the corresponding slot beforehand. The available choices include Jump, Climb, Crouch, Break Rock, and Use Zipline, each suited to a specific obstacle. If the placed card matches the action required at that point, the robot performs the corresponding animation and continues toward the next segment; if it does not, the robot shakes to signal the mistake, an error message is displayed, and it returns to the start of the level so the learner can rearrange the cards and try again. Successfully resolving every decision point on the path completes the level and updates the learner's progress.

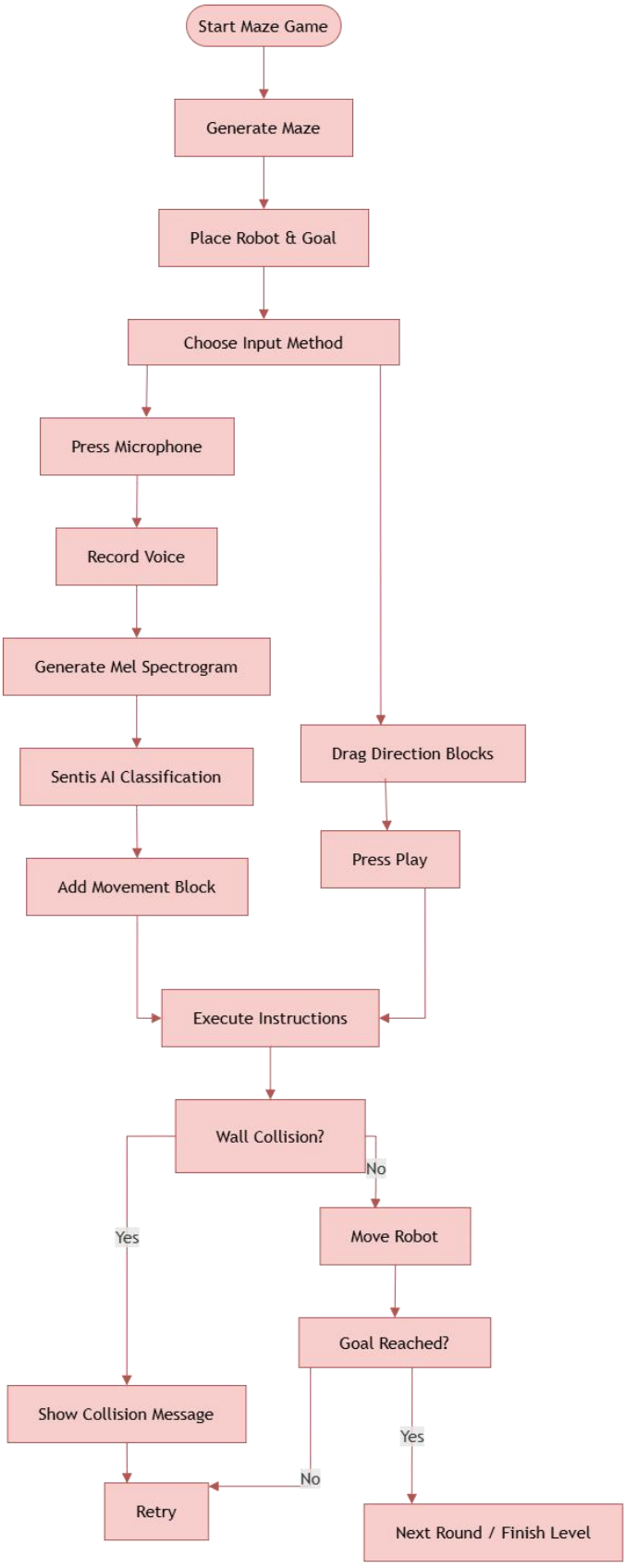

Start Maze Game
Generate Maze
Place Robot & Goal
Choose Input Method
Press Microphone
Record Voice
Generate Mel Spectrogram
Sentis AI Classification
Add Movement Block
Drag Direction Blocks
Press Play
Execute Instructions
Wall Collision?
No
Yes
Move Robot
Goal Reached?
Show Collision Message
Yes
No
Retry
Next Round / Finish Level

Figure 5.1: Activity Workflow AI Decision Making

A flowchart of the AI Decision Making activity, from travelling along a path segment to checking the action card placed at each decision point and either continuing or returning to the start of the level.

5.1.4 Realization

The AI Decision Making activity was developed to demonstrate how intelligent systems select the appropriate action according to a given situation. Internally, each of the six levels is defined as an ordered list of path segments and decision points rather than a free-form grid: the robot always travels automatically between decision points, so the learner's only choices are which action card belongs at each stop. This is a deliberate contrast with the Robot Path Planning activity, where the learner chooses every movement; here, the emphasis shifts entirely onto selecting the right action for a specific obstacle (Jump, Climb, Crouch, Break Rock, or Use Zipline) rather than on navigation itself. An incorrect card sends the robot back to the very start of the level rather than just the failed segment, reinforcing that the whole planned sequence must be coherent before it is executed. After executing the sequence, the application validates the proposed solution, provides immediate visual feedback, and updates the learner's progression upon successful completion of each level.

The following figures illustrate the main interfaces of the AI Decision Making activity.

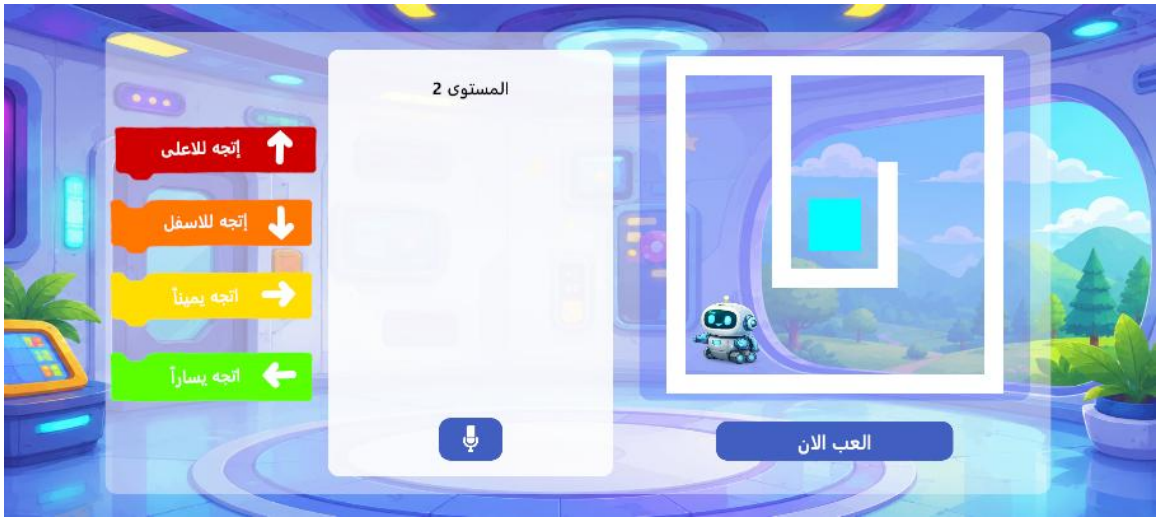


Figure 5.2: AI Decision Making gameplay interface.
The level screen showing the maze view alongside the directional command blocks and the microphone button used for voice input.



Figure 5.3: Action cards arrangement.
The set of action cards, such as Jump, Climb, and Use Zipline, arranged in the order the learner has chosen to solve the level.

5.2 Sprint 8: Game Map

5.2.1 Sprint Overview

The objective of this sprint is to develop the Game Map, which serves as the central navigation interface of the educational application. It connects all educational activities into a unified learning journey while managing player progression and controlling access to unlocked games.

The Game Map visually represents the learner's progress through an animated drone that moves along a predefined path. As each activity is successfully completed, the next game becomes available, encouraging learners to progress through the Artificial Intelligence learning pipeline in a structured and engaging manner.

5.2.2 Sprint Backlog

Table 5.2: Sprint 8 Backlog

No.	Development Tasks	Priority
1	Design the Game Map interface	High
2	Create the navigation path	High
3	Implement robot movement on the map	High
4	Develop the game unlocking system	High
5	Save and load player progression	High

No.	Development Tasks	Priority
6	Display locked and unlocked games	Medium
7	Implement scene navigation	High
8	Integrate all educational activities	High

Lists the development tasks completed during Sprint 8 and their priority.

5.2.3 Activity Workflow

Sprint 8 focused on integrating the Maze Game into the application's progression system. When the player opens the map, the `MapManager` retrieves the saved progress using `PlayerPrefs`, restores the unlocked levels, and places the drone at its latest position.

If the player has just completed a game, a reward panel displaying the completed activity appears. After the player closes the panel, the drone flies toward the next level, the lock animation is played, and the newly unlocked game becomes accessible. The progression is then saved so that it persists between application sessions.

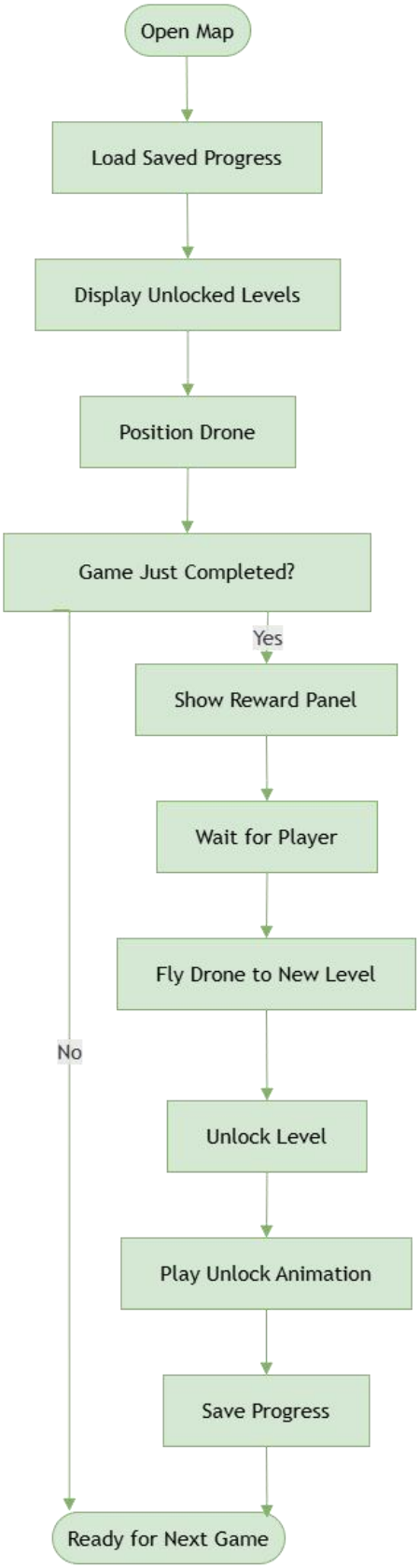

Open Map
Load Saved Progress
Display Unlocked Levels
Position Drone
Game Just Completed?
Yes
No
Show Reward Panel
Wait for Player
Fly Drone to New Level
Unlock Level
Play Unlock Animation
Save Progress
Ready for Next Game

Figure 5.4: Activity Workflow Game Map

A flowchart of the Game Map's progression logic, from restoring the player's saved position to showing the reward panel and unlocking the next activity.

5.2.4 Realization

The Game Map was implemented as the central navigation module of the educational application. Progress is persisted locally through Unity's PlayerPrefs, with a progress-tracking singleton keeping track of each level's status as locked, unlocked, or finished, and the map itself storing which activity the drone last visited so returning players resume exactly where they left off. Finishing an activity's last level triggers a short animated sequence: a reward panel names the completed activity, the drone flies to the next position on the map, and the corresponding lock icon fades out while its level icon transitions from a dimmed grey back to full colour. It is a deliberately celebratory moment intended to make progression feel earned rather than automatic. This centralized system ensures a structured and intuitive learning experience throughout the application.

The following figures illustrate the main interfaces of the Game Map.



Figure 5.5: Locked and unlocked activities.
The overall Game Map, showing the seven mini-game stops connected along a single path, each marked as locked or unlocked.



Figure 5.6: Locked and unlocked levels

A closer view of one activity's level selector, distinguishing completed, unlocked, and still-locked levels.



Figure 5.7: Game Map interface

The Game Map interface with the drone positioned at the learner's current activity.

5.3 UML Modeling

The following UML diagrams present the software architecture of the final modules of the educational application. They illustrate the interactions between the learner and the system during the AI Decision Making activity, as well as the management of navigation and progression through the Game Map.

5.3.1 Use Case Diagram

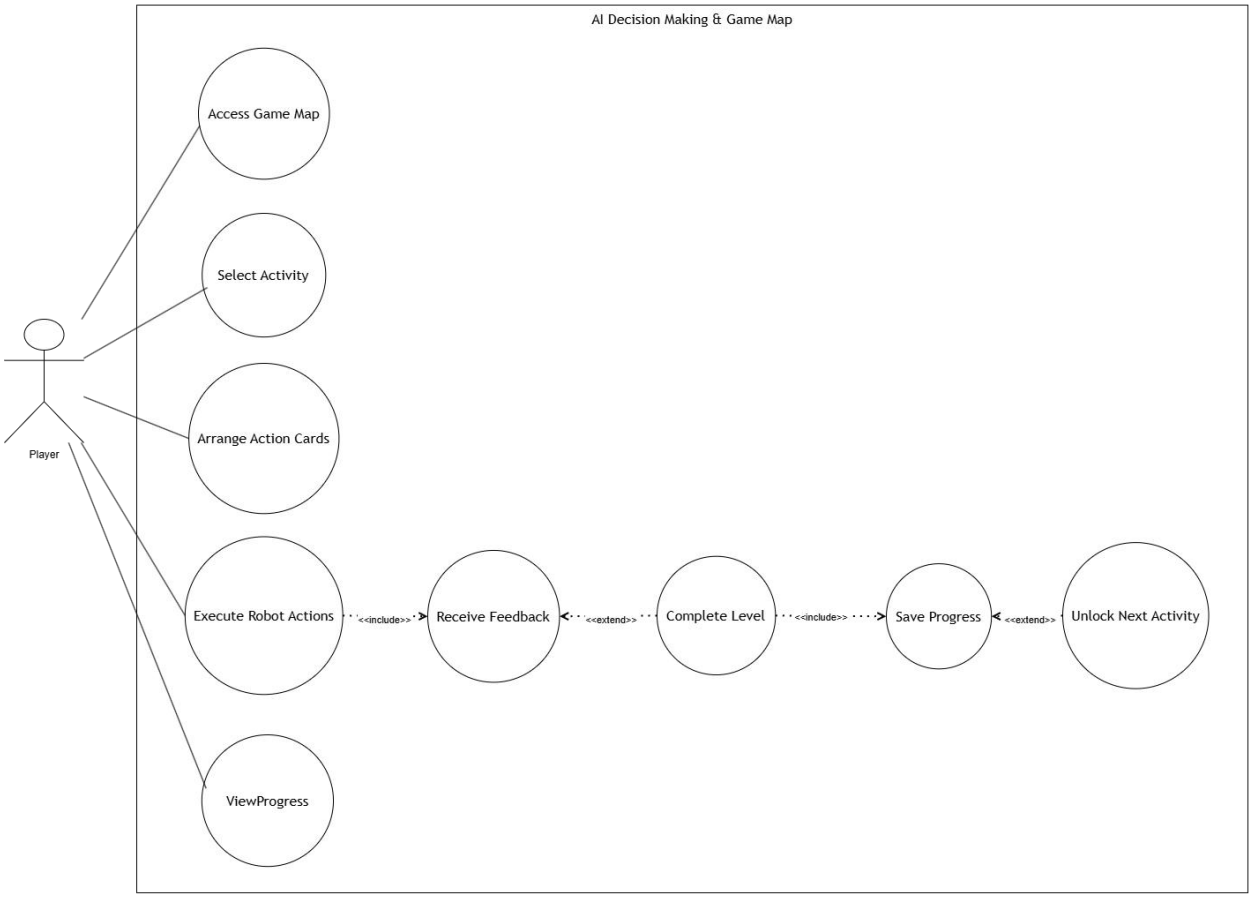


Figure 5.8: Use Case Diagram of the AI Decision Making and Game Map modules.

The same use case pattern as the earlier chapters, adapted for the AI Decision Making activity and the Game Map.

5.3.2 Sequence Diagram

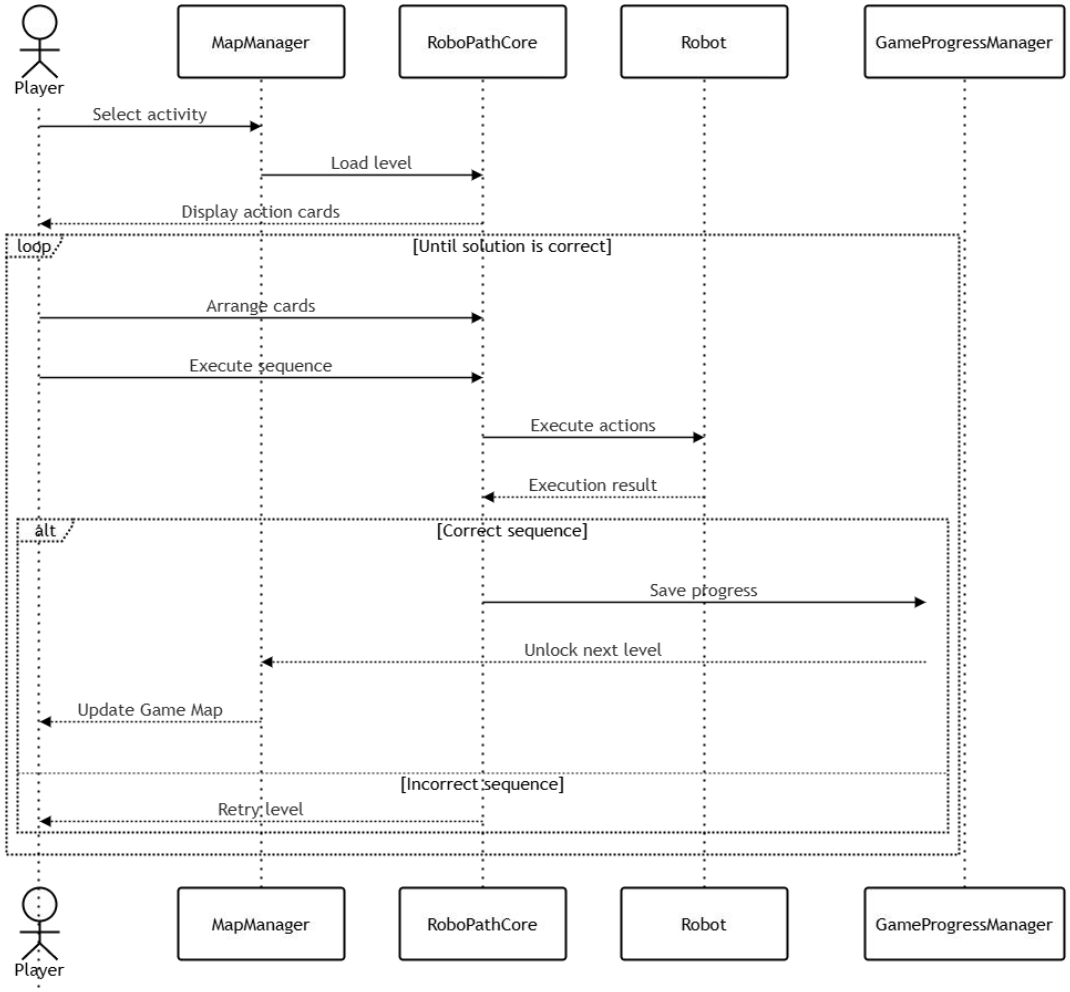


Figure 5.9: Sequence Diagram illustrating the execution flow of the AI Decision Making activity and Game Map navigation.

A sequence diagram tracing the AI Decision Making activity together with the Game Map's navigation and progress-saving flow.

5.3.3 Class Diagram

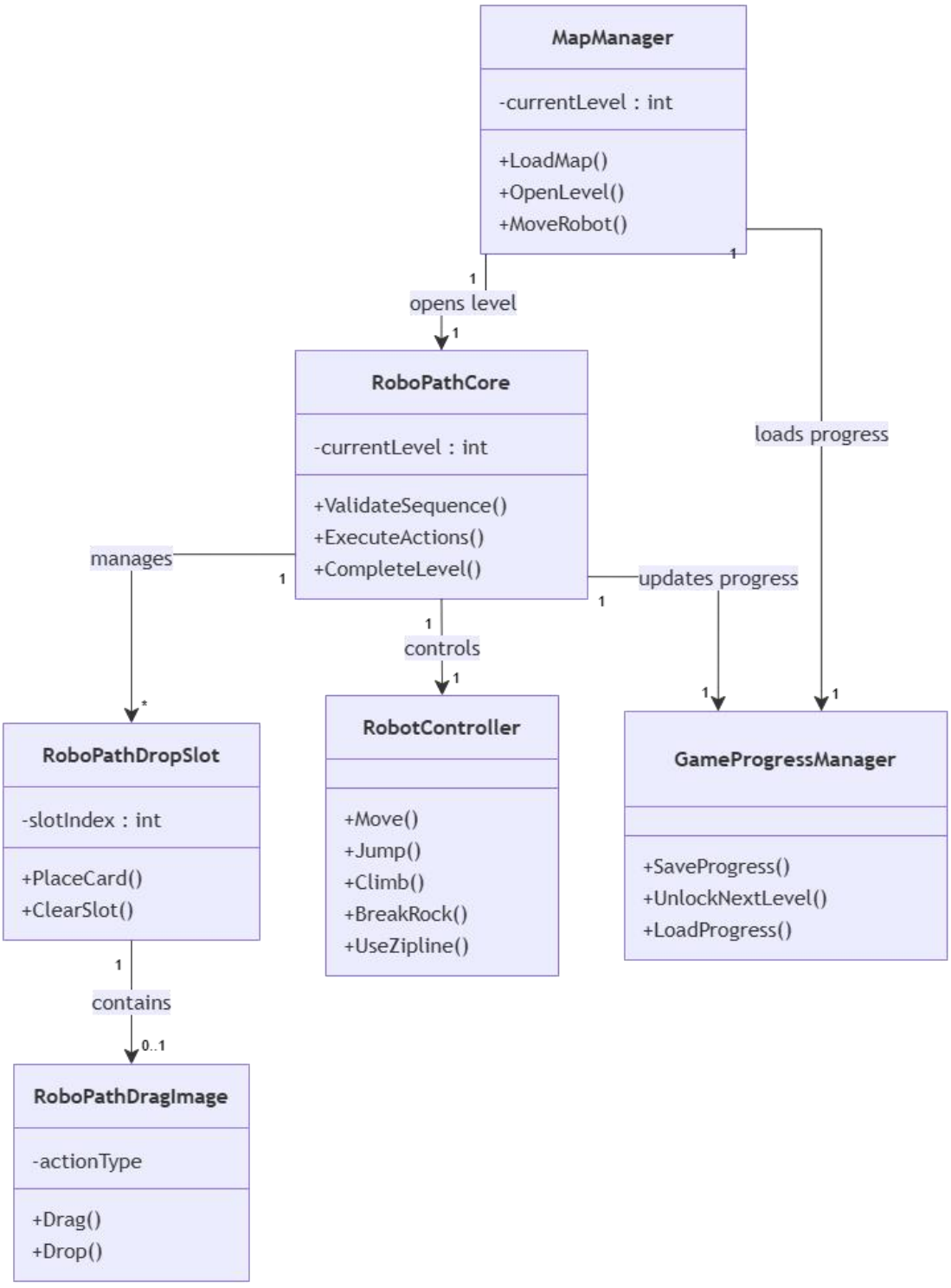


Figure 5.10: Class Diagram of the AI Decision Making and Game Map modules.

A class diagram showing the manager classes shared by the AI Decision Making activity and the Game Map.

Conclusion

This chapter presented the implementation of the final modules of the educational application. The AI Decision Making activity introduced learners to the process of selecting appropriate actions to solve different situations, while the Game Map provided a centralized navigation system that manages learner progression and access to educational activities. Together, these modules complete the educational platform by combining intelligent decision-making concepts with an intuitive progression system, ensuring a coherent and engaging learning experience.

General Conclusion

This Final-Year Project led to the design and development of an educational game that introduces children to the fundamental concepts of Artificial Intelligence through an interactive and engaging learning experience. The proposed solution combines educational content with gamification, enabling learners to progressively discover the different stages of an AI system, including data gathering, data cleaning, image recognition, face detection, intelligent agent behavior, path planning, and decision-making.

From a software engineering perspective, the project was developed following the Agile Scrum methodology, which facilitated effective project planning, continuous development, and incremental delivery of the application. The adoption of modern development tools, including Unity, C#, GitHub, ClickUp, and Visual Studio, contributed to the creation of a robust, modular, and maintainable software architecture.

In addition to the educational mini-games, the project integrates a real Artificial Intelligence component based on speech recognition. A Convolutional Neural Network (CNN) was developed using the PyTorch framework to recognize English voice commands and control the virtual robot within the educational environment. This integration demonstrates the practical application of AI technologies while providing learners with direct interaction with an intelligent system.

The developed game successfully achieves the objectives initially defined by providing an intuitive, interactive, and educational platform adapted to young learners. It offers an innovative approach to introducing Artificial Intelligence concepts while encouraging logical thinking, observation, and problem-solving skills through gameplay.

Despite the satisfactory results obtained, several improvements remain possible. Future work will focus on extending the speech recognition model to support Arabic and French, increasing the diversity of the training dataset using recordings from real users, improving recognition accuracy in noisy environments, and enriching the application with additional educational activities covering more advanced Artificial Intelligence concepts.

In conclusion, this project represents a valuable contribution to the field of educational technology by combining modern software engineering practices with Artificial Intelligence to create an accessible and motivating learning environment. It also provides a solid foundation for future developments aimed at making AI education more interactive, inclusive, and engaging for children.

Webography

[W1] Horizon Education. *Official Website.* Available at: https://feridaroundtheworld.com/ (Accessed: June 2026).

[W2] Unity Technologies. *Unity Documentation.* Available at: https://docs.unity3d.com/ (Accessed: June 2026).

[W3] Microsoft. *C# Documentation.* Available at: https://learn.microsoft.com/dotnet/csharp/ (Accessed: June 2026).

[W4] Microsoft. *Visual Studio Documentation.* Available at: https://learn.microsoft.com/visualstudio/ (Accessed: June 2026).

[W5] PyTorch Foundation. *PyTorch Documentation.* Available at: https://pytorch.org/docs/ (Accessed: June 2026).

[W6] Unity Technologies. *Unity Sentis Documentation.* Available at: https://docs.unity3d.com/Packages/com.unity.sentis/ (Accessed: June 2026).

[W7] Google. *Speech Commands Dataset.* Available at: https://www.tensorflow.org/datasets/catalog/speech_commands (Accessed: June 2026).

[W8] Git Documentation. *Git Reference Manual.* Available at: https://git-scm.com/doc (Accessed: June 2026).

[W9] GitHub. *GitHub Documentation.* Available at: https://docs.github.com/ (Accessed: June 2026).

[W10] ClickUp. *Official Documentation.* Available at: https://docs.clickup.com/ (Accessed: June 2026).

[W11] Slack. *Help Center.* Available at: https://slack.com/help (Accessed: June 2026).

[W12] Krita Foundation. *Krita User Manual.* Available at: https://docs.krita.org/ (Accessed: June 2026).

[W13] Scrum Guides. *The Scrum Guide (2020).* Available at: https://scrumguides.org/ (Accessed: June 2026).

[W14] Machine Learning for Kids. Available at: https://machinelearningforkids.co.uk/ (Accessed: June 2026).

[W15] Code.org. *AI for Oceans.* Available at: https://code.org/oceans (Accessed: June 2026).

[W16] Google. *Quick, Draw!* Available at: https://quickdraw.withgoogle.com/ (Accessed: June 2026).